UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Calculatoare

PROIECT DE DIPLOMĂ

Coordonator științific:
Conf. dr. ing. Nicolae-Alexandru BOTEZATU

Absolvent:
Alin-Ionuț SFICHI

Iași, 2026

UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Calculatoare

Sistem IoT Smart Home bazat pe Bluetooth Mesh

PROIECT DE DIPLOMĂ

Coordonator științific:
Conf. dr. ing. Nicolae-Alexandru BOTEZATU

Absolvent:
Alin-Ionuț SFICHI

Iași, 2026

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII PROIECTULUI DE DIPLOMĂ

Subsemnatul SFICHI ALIN-IONUȚ,
legitimat cu CI seria XV nr. 836019, CNP 5030917330525
autorul lucrării SISTEM IOT SMART HOME BAZAT PE BLUETOOTH MESH

elaborată în vederea susținerii examenului de finalizare a studiilor de licență, programul de studii CALCULATOARE organizat de către Facultatea de Automatică şi Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi" din Iaşi, sesiunea IULIE 2026 a anului universitar 2025-2026, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi" din Iaşi (Manualul Procedurilor, UTI.POM.02 - Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) şi a convențiilor internaționale privind drepturile de autor.

Data
25.06.2026

Semnătura

Cuprins

1 Introducere **1**

2 Fundamentarea teoretică și documentarea bibliografică **3**
2.1 Internet of Things și automatizarea locuințelor . . . 3
2.2 Protocoale wireless utilizate în aplicații IoT . . . 3
2.3 Tehnologia Bluetooth Low Energy . . . 4
2.4 Tehnologia Bluetooth Mesh . . . 4
2.5 Microcontrolerul ESP32-C3 și mediul de dezvoltare ESP-IDF . . . 5
2.6 Rolul gateway-ului și controlul centralizat . . . 5
2.7 Soluții existente în automatizarea locuințelor . . . 6
2.8 Limitări ale soluțiilor existente . . . 6
2.9 Cerințele sistemului propus . . . 7

3 Soluția propusă **9**
3.1 Problema abordată și strategia de rezolvare . . . 9
3.2 Cerințele utilizatorului . . . 9
3.3 Prezentarea generală a sistemului . . . 10
3.4 Arhitectura sistemului . . . 11
3.4.1 Comunicarea în rețeaua Bluetooth Mesh . . . 11
3.4.2 Modelul publish–subscribe . . . 12
3.4.3 Arhitectura gateway-ului . . . 13
3.4.4 Scenarii de automatizare . . . 14
3.4.5 Avantajele arhitecturii alese . . . 14
3.4.6 Procesul de provizionare și configurare a nodurilor . . . 14
3.5 Platforma hardware . . . 16
3.5.1 Microcontrolerul ESP32-C3 . . . 17
3.5.2 Tipuri de plăci utilizate în proiect . . . 17
3.5.3 Nodul pentru controlul iluminatului . . . 18
3.5.4 Nodul întrerupător . . . 19
3.5.5 Nodul IR . . . 19
3.5.6 Nodul senzor . . . 21
3.5.7 Alimentarea nodurilor . . . 23
3.5.8 Avantaje și dezavantaje ale platformei hardware . . . 23
3.6 Tehnologii software utilizate . . . 24
3.6.1 Mediul de dezvoltare ESP-IDF . . . 25
3.6.2 Stiva Bluetooth Mesh . . . 26
3.6.3 Structura firmware-ului nodurilor . . . 27
3.6.4 Software-ul gateway-ului . . . 28
3.6.5 Protocolul MQTT . . . 29
3.6.6 Aplicația Flask . . . 29

3.6.7 Implementarea scenariilor de automatizare 30
3.6.8 Avantaje și dezavantaje ale tehnologiilor alese 30
3.7 Implementarea soluției 31
3.7.1 Nod întrerupător 31
3.7.2 Nod pentru controlul iluminatului 32
3.7.3 Nod IR (telecomandă universală) 35
3.7.4 Nod senzor temperatură și umiditate 38
3.7.5 Nod gateway și aplicația de control 41
3.7.6 Probleme întâmpinate și soluții 45

4 Testarea aplicației și rezultate experimentale **47**
4.1 Punerea în funcțiune a sistemului 47
4.2 Testarea funcțională a nodurilor 48
4.3 Testarea rețelei Bluetooth Mesh 49
4.4 Testarea comunicării prin gateway și MQTT 49
4.5 Testarea scenariilor de automatizare 50
4.6 Încărcarea procesorului și utilizarea memoriei 50
4.7 Limitări ale transmisiei și latență 51
4.8 Fiabilitate, securitate și scalabilitate 52
4.9 Concluzii privind testarea 52

5 Concluzii **53**

Bibliografie **55**

Sistem IoT Smart Home bazat pe Bluetooth Mesh

Alin-Ionuț SFICHI

Rezumat

Lucrarea prezintă proiectarea și implementarea unui sistem domotic (*smart home*) bazat pe tehnologia Bluetooth Mesh, utilizând microcontrolere ESP32-C3 pentru realizarea unei rețele de dispozitive inteligente [1]. Sistemul este alcătuit din mai multe tipuri de noduri cu funcționalități diferite, precum noduri întrerupător pentru trimiterea comenzilor, noduri pentru controlul iluminatului, noduri IR pentru controlul dispozitivelor electronice și noduri senzor pentru monitorizarea parametrilor de mediu [2]. Acestea sunt integrate într-o arhitectură *mesh* care permite extinderea rețelei și creșterea fiabilității prin retransmiterea mesajelor între noduri [3, 4].

Pentru configurarea și controlul centralizat al sistemului a fost realizat un nod gateway bazat pe ESP32-C3, conectat la o aplicație software care permite monitorizarea și controlul dispozitivelor din rețea [5]. Interfața oferă posibilitatea creării unor scenarii de automatizare, prin care mai multe acțiuni pot fi executate simultan, cum ar fi pornirea iluminatului, transmiterea unei comenzi infraroșu sau modificarea stării unor noduri în funcție de condiții definite de utilizator. Comunicarea între noduri și gateway utilizează modelul *publish–subscribe* specific protocolului Bluetooth Mesh [3], iar pentru implementare au fost folosite modelele SIG standard [6], extinse acolo unde a fost necesar.

Firmware-ul nodurilor a fost dezvoltat în mediul ESP-IDF utilizând limbajul C/C++ [5], iar procesul de *provizionare* a inclus configurarea adreselor, a cheilor de securitate și a parametrilor rețelei [3]. Testarea sistemului a fost realizată în mai multe scenarii de utilizare, incluzând controlul iluminatului cu intensitate variabilă, transmiterea comenzilor IR [7] către dispozitive externe și execuția unor scenarii de automatizare definite în interfața software. Rezultatele experimentale au demonstrat stabilitatea comunicației, timpi de răspuns reduși și posibilitatea extinderii rețelei fără modificări majore ale infrastructurii hardware.

Soluția propusă demonstrează că utilizarea tehnologiei Bluetooth Mesh [4] împreună cu microcontrolere ESP32-C3 [2] permite realizarea unui sistem domotic flexibil, extensibil și eficient, capabil să ofere control centralizat și automatizări complexe folosind componente cu cost redus.

Cuvinte-cheie: IoT, Bluetooth Mesh, ESP32-C3, smart home, gateway, automatizare, ESP-IDF.

Capitolul 1. Introducere

În ultimii ani, dezvoltarea rapidă a tehnologiilor din domeniul sistemelor embedded și al comunicațiilor wireless a condus la apariția unui număr tot mai mare de aplicații bazate pe conceptul Internet of Things (IoT) [1]. Acest concept presupune interconectarea unor dispozitive electronice inteligente care pot colecta, transmite și procesa informații fără intervenție umană permanentă. Domeniul IoT este utilizat într-o varietate largă de aplicații, precum automatizarea industrială, monitorizarea mediului, sistemele medicale, transportul inteligent și automatizarea locuințelor [1]. Dintre acestea, sistemele de tip smart home reprezintă una dintre cele mai răspândite aplicații, datorită avantajelor oferite în ceea ce privește confortul, eficiența energetică și posibilitatea controlului de la distanță.

Conceptul de locuință inteligentă presupune integrarea unor senzori, actuatori și unități de control într-o rețea locală, astfel încât utilizatorul să poată monitoriza și controla diverse echipamente electrice. Exemple de astfel de echipamente sunt sistemele de iluminat, prizele inteligente, aparatele de climatizare, televizoarele sau sistemele de securitate. Pentru realizarea unei astfel de rețele este necesară utilizarea unui protocol de comunicație fără fir care să asigure consum redus de energie, stabilitate în transmiterea datelor și posibilitatea extinderii sistemului prin adăugarea de noi dispozitive.

În aplicațiile domotice sunt utilizate mai multe tehnologii de comunicație, dintre care cele mai cunoscute sunt Wi-Fi, ZigBee, Z-Wave și Bluetooth Low Energy [8]. Wi-Fi oferă viteze mari de transfer și conectivitate directă la internet, însă consumul de energie este ridicat, ceea ce îl face mai puțin potrivit pentru dispozitive alimentate la baterie. Protocoalele ZigBee și Z-Wave sunt proiectate special pentru aplicații IoT și permit realizarea unor rețele de tip mesh, dar necesită hardware dedicat și nu sunt întotdeauna ușor de integrat în sisteme cu cost redus [8]. Bluetooth Low Energy reprezintă o alternativă atractivă datorită consumului scăzut de energie și disponibilității pe un număr mare de microcontrolere moderne [4].

O limitare a comunicației Bluetooth clasice este faptul că aceasta este bazată în principal pe conexiuni punct-la-punct sau pe o arhitectură de tip master-slave. Pentru aplicațiile în care există mai multe dispozitive distribuite în spațiu, această abordare nu este suficient de flexibilă. Pentru a rezolva această problemă a fost introdus standardul Bluetooth Mesh [3], care extinde funcționalitatea Bluetooth Low Energy prin introducerea unei arhitecturi de rețea de tip *mesh*. Într-o rețea mesh, fiecare nod poate retransmite mesajele primite, ceea ce permite extinderea ariei de acoperire și creșterea fiabilității comunicației [4]. Această caracteristică face ca Bluetooth Mesh să fie potrivit pentru aplicații de automatizare a locuințelor și a clădirilor inteligente, unde este necesară comunicarea între un număr mare de dispozitive [3].

În cadrul unui sistem domotic modern, pe lângă nodurile care execută efectiv comenzile, este necesară și existența unei componente de control centralizat. Aceasta poate fi realizată sub forma unui gateway care permite configurarea rețelei, monitorizarea dispozitivelor și definirea unor scenarii de automatizare. Prin utilizarea unei interfețe software, utilizatorul poate crea reguli prin care mai multe acțiuni sunt executate simultan, de exemplu aprinderea iluminatului, transmiterea unei comenzi către un aparat controlat prin infraroșu sau modificarea stării unor dispozitive în funcție de valori citite de senzori [9]. Implementarea unui gateway oferă flexibilitate sistemului și permite extinderea acestuia cu funcționalități suplimentare fără modificări majore în structura nodurilor existente.

Lucrarea de față are ca obiectiv proiectarea și implementarea unui sistem domotic bazat pe tehnologia Bluetooth Mesh, utilizând microcontrolere ESP32-C3 [2]. Alegerea acestei platforme a fost determinată de faptul că oferă suport hardware pentru Bluetooth Low Energy, are consum redus de energie și poate fi programată folosind mediul de dezvoltare ESP-IDF [5], care include suport pentru implementarea rețelelor mesh. În cadrul proiectului au fost realizate mai multe tipuri de noduri, fiecare având un rol specific în rețea. Au fost dezvoltate noduri întrerupător pentru trimiterea comenzilor, noduri de putere pentru controlul iluminatului, noduri infraroșu pentru transmiterea comenzilor

către dispozitive externe și noduri senzor pentru monitorizarea parametrilor de mediu. De asemenea, a fost implementat un nod gateway care permite configurarea și controlul sistemului prin intermediul unei interfețe software.

Obiectivul general al lucrării este realizarea unei rețele funcționale de tip Bluetooth Mesh care să permită controlul centralizat și automatizarea unor echipamente electrice dintr-o locuință [3]. Pentru atingerea acestui obiectiv au fost stabilite mai multe obiective specifice, printre care studierea protocolului Bluetooth Mesh [3, 4], proiectarea arhitecturii sistemului, implementarea firmware-ului pentru noduri [5], realizarea conexiunilor hardware și testarea funcționării în diferite scenarii de utilizare. Un alt obiectiv important a fost realizarea unui mecanism de control prin gateway și interfață software, care să permită definirea unor scenarii de automatizare și extinderea sistemului fără modificări majore ale codului existent.

Metodologia utilizată în realizarea proiectului a inclus atât documentarea teoretică, cât și dezvoltarea practică a sistemului. În prima etapă au fost studiate documentațiile oficiale pentru Bluetooth Mesh [3] și ESP32-C3 [2], precum și soluțiile existente în domeniul sistemelor domotice. În etapa următoare a fost proiectată structura generală a rețelei și au fost definite rolurile fiecărui tip de nod. Implementarea software a fost realizată în limbajul C utilizând framework-ul ESP-IDF [5], iar partea hardware a fost construită folosind module ESP32-C3 și componente electronice specifice fiecărei funcționalități. După realizarea nodurilor individuale, sistemul a fost testat în mediu de laborator pentru a verifica stabilitatea comunicației, timpul de răspuns și funcționarea scenariilor de automatizare.

Lucrarea este structurată în cinci capitole. Capitolul al doilea prezintă fundamentarea teoretică și analiza soluțiilor existente în domeniul sistemelor domotice și al rețelelor Bluetooth Mesh. Capitolul al treilea descrie soluția propusă, incluzând arhitectura sistemului, alegerea componentelor hardware și implementarea software. Capitolul al patrulea prezintă modul de testare al sistemului și rezultatele experimentale obținute. În final, capitolul al cincilea conține concluziile lucrării și posibile direcții de dezvoltare ulterioară.

Capitolul 2. Fundamentarea teoretică și documentarea bibliografică

2.1. Internet of Things și automatizarea locuințelor

Internet of Things (IoT) reprezintă un concept modern care descrie interconectarea unui număr mare de dispozitive electronice capabile să comunice între ele prin intermediul rețelelor de date [1]. Aceste dispozitive pot include senzori, actuatori, microcontrolere sau sisteme embedded care colectează informații din mediul înconjurător și transmit date către alte echipamente pentru procesare sau control. În ultimii ani, tehnologia IoT a fost adoptată într-o varietate largă de domenii, precum automatizarea industrială, monitorizarea mediului, sistemele medicale, transportul inteligent și automatizarea locuințelor [1].

Unul dintre cele mai răspândite domenii de aplicare ale IoT este automatizarea rezidențială, cunoscută și sub denumirea de smart home. Un sistem smart home permite controlul și monitorizarea dispozitivelor electrice dintr-o locuință folosind o rețea de comunicație fără fir. Într-o astfel de rețea sunt integrați senzori, module de comandă și unități de control, care pot funcționa independent sau pot fi coordonate de un sistem central. Utilizatorul poate controla dispozitivele prin aplicații software, interfețe web, telecomenzi sau comenzi vocale.

Implementarea unui sistem de automatizare pentru locuințe oferă mai multe avantaje. Printre cele mai importante se numără creșterea confortului, reducerea consumului de energie și îmbunătățirea siguranței. De exemplu, iluminatul poate fi pornit automat la detectarea prezenței, temperatura poate fi reglată în funcție de condițiile din cameră, iar anumite aparate pot fi oprite automat atunci când nu sunt utilizate. În plus, sistemele moderne permit definirea unor scenarii de automatizare, în care mai multe acțiuni sunt executate simultan în funcție de condiții stabilite de utilizator.

Pentru realizarea unui sistem smart home este necesară alegerea unei tehnologii de comunicație potrivite. Dispozitivele trebuie să comunice între ele într-un mod stabil, cu consum redus de energie și cu posibilitatea extinderii rețelei [8]. În funcție de aplicație, se pot utiliza mai multe protocoale wireless, fiecare având avantaje și dezavantaje. Alegerea protocolului influențează în mod direct performanța sistemului, costul hardware și complexitatea implementării.

2.2. Protocoale wireless utilizate în aplicații IoT

În aplicațiile IoT sunt utilizate mai multe tehnologii de comunicație fără fir, dintre care cele mai răspândite sunt Wi-Fi, ZigBee, Z-Wave și Bluetooth Low Energy [8]. Fiecare dintre aceste protocoale a fost proiectat pentru anumite tipuri de aplicații și prezintă caracteristici diferite în ceea ce privește consumul de energie, viteza de transfer și distanța de comunicare.

Wi-Fi este una dintre cele mai utilizate tehnologii datorită vitezei mari de transfer și a compatibilității cu rețelele existente. Avantajul principal este posibilitatea conectării directe la internet, fără a fi necesare dispozitive suplimentare. Totuși, consumul de energie este relativ ridicat, ceea ce face ca Wi-Fi să fie mai puțin potrivit pentru dispozitive alimentate la baterie sau pentru rețele cu un număr mare de noduri [8].

ZigBee este un protocol proiectat special pentru aplicații IoT și automatizarea locuințelor. Acesta permite realizarea unor rețele de tip mesh, în care fiecare nod poate retransmite mesajele primite către alte noduri. Consumul de energie este redus, iar stabilitatea comunicației este ridicată. Dezavantajele principale sunt viteza mică de transfer și necesitatea unor module hardware dedicate, care pot crește costul sistemului [8].

Z-Wave este o altă tehnologie utilizată în sistemele domotice comerciale. Aceasta oferă comunicație fiabilă și consum redus de energie, însă este o tehnologie proprietară și necesită dispozitive compatibile cu standardul respectiv. Din acest motiv, flexibilitatea în dezvoltarea unor soluții personalizate este limitată.

Bluetooth Low Energy reprezintă o alternativă atractivă pentru aplicațiile IoT datorită consumului redus de energie și a suportului larg pe microcontrolere moderne [4]. În versiunea sa standard, Bluetooth este utilizat pentru comunicații punct-la-punct, însă versiunile recente introduc suport pentru rețele distribuite prin intermediul standardului Bluetooth Mesh [3]. Această extensie permite realizarea unor rețele cu mai multe noduri fără a fi necesară o infrastructură complexă.

În aplicațiile de automatizare a locuințelor este important ca protocolul ales să permită extinderea sistemului, să ofere stabilitate în transmiterea datelor și să nu necesite consum mare de energie. Din acest motiv, tehnologia Bluetooth Mesh reprezintă o soluție potrivită pentru realizarea unei rețele distribuite de dispozitive inteligente [3, 4].

2.3. Tehnologia Bluetooth Low Energy

Bluetooth Low Energy (BLE) este o tehnologie de comunicație wireless introdusă în specificația Bluetooth 4.0, proiectată pentru aplicații în care consumul de energie trebuie să fie redus [4]. Spre deosebire de Bluetooth Classic, care a fost conceput în principal pentru transfer de date la viteze mai mari, BLE este optimizat pentru transmiterea unor cantități mici de date la intervale regulate, ceea ce îl face potrivit pentru senzori, dispozitive portabile și sisteme IoT.

Principiul de funcționare al BLE se bazează pe existența a două tipuri principale de dispozitive: central și periferic. Dispozitivul central inițiază conexiunea și controlează comunicarea, iar dispozitivul periferic răspunde la cereri și transmite date atunci când este solicitat. Această arhitectură este suficientă pentru aplicații simple, însă devine limitată atunci când este necesară comunicarea între un număr mare de dispozitive distribuite într-un spațiu extins.

Un alt aspect important al BLE este consumul redus de energie [4]. Dispozitivele pot intra în moduri de economisire a energiei și pot transmite date doar atunci când este necesar. Acest lucru permite utilizarea bateriilor pentru perioade lungi de timp, ceea ce reprezintă un avantaj important pentru aplicațiile IoT. Cu toate acestea, în forma sa standard, BLE nu oferă suport pentru rețele de tip *mesh*, iar distanța de comunicare este limitată.

Pentru a depăși aceste limitări a fost introdus standardul Bluetooth Mesh [3], care extinde funcționalitatea Bluetooth Low Energy și permite realizarea unor rețele cu mai multe noduri, capabile să comunice între ele fără a fi necesară o conexiune directă între fiecare dispozitiv.

2.4. Tehnologia Bluetooth Mesh

Bluetooth Mesh este o extensie a standardului Bluetooth Low Energy care permite realizarea unor rețele de tip mesh, în care dispozitivele pot comunica folosind un mecanism multi-hop [3]. Într-o astfel de rețea, fiecare nod poate retransmite mesajele primite către alte noduri, ceea ce permite extinderea ariei de acoperire și creșterea fiabilității comunicației [4]. Această arhitectură este potrivită pentru aplicații în care există un număr mare de dispozitive, cum ar fi sistemele de iluminat inteligent sau automatizarea clădirilor.

Spre deosebire de comunicația punct-la-punct, Bluetooth Mesh utilizează un model de transmitere bazat pe mecanismul publish–subscribe [3]. În acest model, un nod poate publica un mesaj către o anumită adresă, iar toate nodurile care sunt abonate la acea adresă pot recepționa mesajul. Acest mod de funcționare permite controlul simultan al mai multor dispozitive fără a fi necesară trimiterea unei comenzi separate pentru fiecare nod.

Arhitectura Bluetooth Mesh introduce conceptul de nod, element și model [3]. Un nod reprezintă un dispozitiv fizic din rețea, care poate conține unul sau mai multe elemente. Fiecare element poate implementa unul sau mai multe modele [6], care definesc comportamentul dispozitivului. De exemplu, un model Generic OnOff permite controlul stării unui releu sau al unei surse de lumină, iar un model de tip senzor permite transmiterea valorilor măsurate către alte noduri.

Pentru ca un dispozitiv să devină parte din rețea, acesta trebuie să fie provizionat. Procesul de provizionare reprezintă configurarea inițială a nodului, în timpul căreia sunt atribuite adresele de rețea și cheile de securitate necesare pentru comunicare [3]. Bluetooth Mesh utilizează criptare pentru

toate mesajele, ceea ce asigură securitatea datelor transmise între noduri [3].

Standardul definește mai multe tipuri de roluri pe care un nod le poate avea în rețea [3]. Nodurile de tip Relay pot retransmite mesajele primite pentru a extinde aria de acoperire. Nodurile de tip Proxy permit conectarea unor dispozitive care nu suportă direct protocolul mesh, cum ar fi telefoanele mobile. Nodurile de tip Friend și Low Power sunt utilizate pentru reducerea consumului de energie, permițând unor dispozitive să intre în modul de repaus și să primească mesajele prin intermediul unui nod prieten.

Un avantaj important al rețelelor mesh este toleranța la erori [4]. Dacă un nod nu mai funcționează, mesajele pot fi retransmise prin intermediul altor noduri, fără a fi necesară reconfigurarea întregului sistem. Acest lucru face ca Bluetooth Mesh să fie potrivit pentru aplicații în care fiabilitatea este importantă.

Datorită mecanismului de retransmitere controlată (Managed Flooding) [10], rețelele Bluetooth Mesh pot conține un număr mare de dispozitive, chiar și mii de noduri într-o singură rețea, fără a fi necesară o infrastructură centralizată complexă. Această caracteristică face ca tehnologia să fie potrivită pentru sisteme de automatizare a locuințelor, unde dispozitivele sunt distribuite în mai multe camere sau pe mai multe etaje [3].

În aplicațiile domotice, Bluetooth Mesh permite realizarea unor sisteme flexibile, în care nodurile pot fi adăugate sau eliminate fără modificări majore în restul rețelei. De asemenea, mecanismul publish–subscribe permite definirea unor scenarii de automatizare, în care mai multe dispozitive reacționează la aceeași comandă sau la același eveniment [6].

2.5. Microcontrolerul ESP32-C3 și mediul de dezvoltare ESP-IDF

Pentru implementarea nodurilor din rețeaua propusă a fost ales microcontrolerul ESP32-C3, produs de compania Espressif Systems [2]. Acest microcontroler face parte din familia ESP32 și este proiectat pentru aplicații IoT, oferind conectivitate wireless integrată și resurse hardware suficiente pentru aplicații embedded complexe. ESP32-C3 utilizează un nucleu RISC-V de 32 de biți [11], care poate funcționa la frecvențe de până la 160 MHz, și dispune de memorie RAM internă, memorie flash externă și multiple interfețe periferice [2].

Un avantaj important al microcontrolerului ESP32-C3 este suportul hardware pentru Bluetooth Low Energy și Bluetooth Mesh, ceea ce permite implementarea directă a rețelelor mesh fără a fi necesare module suplimentare [2]. De asemenea, microcontrolerul include acceleratoare hardware pentru criptografie, ceea ce este util pentru implementarea mecanismelor de securitate necesare în comunicația mesh [3].

ESP32-C3 dispune mai multe interfețe periferice, cum ar fi GPIO, UART, SPI, I2C și PWM, care permit conectarea unor senzori și actuatori diferiți [2]. Acest lucru face posibilă realizarea unor noduri cu funcționalități variate, precum controlul iluminatului, citirea valorilor de la senzori sau transmiterea de comenzi către alte dispozitive.

Pentru dezvoltarea firmware-ului a fost utilizat mediul ESP-IDF (Espressif IoT Development Framework) [5], care este framework-ul oficial pentru programarea microcontrolerelor ESP32. ESP-IDF oferă biblioteci pentru comunicație wireless, controlul perifericelor și implementarea protocolului Bluetooth Mesh [5]. Utilizarea acestui mediu permite accesul la toate funcțiile hardware și oferă flexibilitate în dezvoltarea aplicației. Execuția task-urilor se bazează pe sistemul de operare în timp real FreeRTOS, integrat în ESP-IDF [12].

2.6. Rolul gateway-ului și controlul centralizat

Într-un sistem domotic complex, pe lângă nodurile care execută efectiv comenzile, este necesară și existența unei componente care să permită configurarea și controlul întregii rețele. Această componentă este realizată sub forma unui gateway, care face legătura între rețeaua mesh și aplicația utilizatorului.

Gateway-ul are rolul de a inițializa rețeaua, de a realiza procesul de provizionare al nodurilor

și de a transmite comenzile către dispozitivele din rețea. Prin intermediul gateway-ului, utilizatorul poate vizualiza starea nodurilor, poate trimite comenzi și poate defini reguli de automatizare [3]. În sistemul propus, gateway-ul este realizat tot folosind un microcontroler ESP32-C3 [2], configurat pentru a comunica atât cu rețeaua mesh, cât și cu aplicația de control.

Utilizarea unui gateway permite controlul centralizat al sistemului și oferă posibilitatea realizării unei interfețe grafice pentru utilizator. Prin intermediul acestei interfețe pot fi create scenarii de automatizare, în care mai multe acțiuni sunt executate simultan. De exemplu, se poate defini un scenariu în care aprinderea unui întrerupător determină pornirea iluminatului, transmiterea unei comenzi infraroșu [7] către televizor și activarea unui alt dispozitiv din rețea.

Scenariile de automatizare reprezintă o funcționalitate importantă în sistemele smart home moderne, deoarece permit utilizatorului să controleze mai multe dispozitive printr-o singură comandă [1]. Implementarea acestora necesită un mecanism de control centralizat și posibilitatea de transmitere a comenzilor către mai multe noduri simultan, lucru care poate fi realizat eficient folosind modelul publish–subscribe din Bluetooth Mesh [3].

2.7. Soluții existente în automatizarea locuințelor

Pe piață există numeroase soluții comerciale pentru automatizarea locuințelor, dezvoltate de companii precum Philips, Samsung, Google sau Amazon. Aceste sisteme permit controlul iluminatului, al prizelor inteligente și al altor echipamente prin aplicații mobile sau interfețe web. Majoritatea acestor soluții utilizează protocoale wireless precum Wi-Fi, ZigBee sau Z-Wave [8].

Sistemele bazate pe Wi-Fi sunt ușor de integrat în rețelele existente și oferă viteze mari de transfer, însă consumul de energie este ridicat, iar stabilitatea depinde de routerul local sau de conexiunea la internet. În multe cazuri, dispozitivele trebuie să comunice printr-un server extern, ceea ce poate introduce întârzieri și dependență de servicii cloud [1].

Soluțiile bazate pe ZigBee sau Z-Wave utilizează rețele de tip mesh și oferă consum redus de energie, însă necesită hub-uri dedicate și dispozitive compatibile cu standardul respectiv [8]. În plus, unele sisteme sunt proprietare, ceea ce limitează posibilitatea dezvoltării unor aplicații personalizate.

Există și soluții bazate pe platforme open-source, care folosesc microcontrolere precum Arduino, ESP8266 sau ESP32 [5]. Acestea oferă flexibilitate mare, însă implementarea unei rețele stabile și securizate poate fi dificilă fără utilizarea unui protocol standardizat.

Bluetooth Mesh reprezintă o alternativă modernă [3], deoarece permite realizarea unor rețele distribuite folosind hardware accesibil și fără a fi necesară o infrastructură complexă. În plus, suportul pentru criptare și mecanismul publish–subscribe permit realizarea unor sisteme sigure și scalabile [3, 4].

2.8. Limitări ale soluțiilor existente

Analiza soluțiilor disponibile arată că multe dintre acestea prezintă anumite limitări [1]. Unele sisteme depind de conexiunea la internet pentru funcționare, ceea ce poate afecta fiabilitatea în cazul întreruperii conexiunii. Alte soluții necesită echipamente dedicate costisitoare sau nu permit modificarea funcționalității de către utilizator.

În cazul soluțiilor bazate pe Wi-Fi, consumul de energie este relativ mare, ceea ce face dificilă utilizarea în dispozitive alimentate la baterie [8]. Protocoalele proprietare pot limita interoperabilitatea între dispozitive de la producători diferiți. De asemenea, unele sisteme nu permit extinderea ușoară prin adăugarea de noi noduri fără reconfigurarea întregii rețele.

Din aceste motive este necesară dezvoltarea unei soluții care să fie flexibilă, extensibilă și independentă de infrastructuri externe. Utilizarea unei rețele Bluetooth Mesh permite realizarea unui sistem distribuit, în care nodurile pot comunica direct între ele, fără a depinde de un server central [3, 4].

2.9. Cerințele sistemului propus

Pe baza analizei teoretice și a soluțiilor existente au fost stabilite cerințele principale pentru sistemul propus. În primul rând, sistemul trebuie să permită comunicarea între mai multe dispozitive folosind o rețea wireless stabilă și sigură [3]. În al doilea rând, trebuie să fie posibilă extinderea rețelei prin adăugarea de noi noduri fără modificări majore ale software-ului.

Un alt obiectiv important este realizarea unui control centralizat prin intermediul unui gateway și al unei interfețe software, care să permită monitorizarea și configurarea dispozitivelor [9]. Sistemul trebuie să permită definirea unor scenarii de automatizare și transmiterea comenzilor către mai multe noduri simultan.

De asemenea, este necesar ca soluția să utilizeze componente hardware accesibile [2] și să aibă consum redus de energie, astfel încât să poată fi utilizată în aplicații reale. Implementarea trebuie să fie realizată folosind un protocol standardizat [3], care să ofere securitate și stabilitate în transmiterea datelor.

Pe baza acestor cerințe a fost aleasă implementarea unei rețele domotice bazate pe Bluetooth Mesh [3] și microcontrolere ESP32-C3 [2], soluție care oferă flexibilitate, cost redus și posibilitatea dezvoltării ulterioare a sistemului.

Capitolul 3. Soluția propusă

3.1. Problema abordată și strategia de rezolvare

În domeniul automatizării locuințelor există numeroase soluții comerciale, însă multe dintre acestea sunt sisteme închise, dependente de infrastructuri cloud sau de echipamente dedicate costisitoare [1]. În plus, unele soluții nu permit integrarea facilă a unor dispozitive personalizate și nu oferă flexibilitatea necesară pentru dezvoltarea unor scenarii complexe de automatizare. Din aceste motive, s-a considerat necesar realizarea unui sistem domotic modular, extensibil și independent, care să poată fi configurat și extins de utilizator fără modificări majore ale infrastructurii existente.

Problema principală pe care o rezolvă soluția propusă este realizarea unei rețele de dispozitive inteligente care să poată comunica între ele fără a depinde de un router Wi-Fi sau de un server extern, dar care să permită în același timp control centralizat printr-o interfață software. Sistemul trebuie să permită controlul iluminatului, transmiterea de comenzi către dispozitive controlate prin infraroșu [7], citirea valorilor de la senzori și definirea unor scenarii de automatizare configurabile din interfață.

Pentru rezolvarea acestei probleme s-a ales utilizarea protocolului Bluetooth Mesh [3], care permite realizarea unei rețele distribuite în care fiecare nod poate retransmite mesajele celorlalte noduri [10]. În acest mod se obține o acoperire mai mare și o toleranță ridicată la erori [4], deoarece sistemul nu depinde de un singur punct de control.

Strategia de implementare a sistemului a fost următoarea:

- utilizarea microcontrolerelor ESP32-C3 [2] pentru toate nodurile din rețea, datorită suportului pentru Bluetooth Low Energy și Bluetooth Mesh și a prețului accesibil
- implementarea mai multor tipuri de noduri, fiecare cu rol specific
- utilizarea unui gateway pentru conectarea rețelei mesh la o aplicație software
- realizarea unei aplicații de control în Python, integrând interfața grafică sub forma unui site WEB cu ajutorul Flask [13] și comunicarea cu gateway-ul prin intermediul MQTT [9]
- utilizarea unui broker Mosquitto [14] pentru schimbul de mesaje între aplicație și gateway
- implementarea unui mecanism de scenarii de automatizare configurabile din interfață
- proiectarea unor circuite hardware simple și reproductibile pentru fiecare nod

Prin această abordare se obține un sistem flexibil, care poate fi extins prin adăugarea de noi noduri fără modificarea celor existente [3], care permite controlul atât local, cât și de la distanță.

3.2. Cerințele utilizatorului

Înaintea proiectării sistemului au fost definite cerințele principale pe care trebuie să le îndeplinească aplicația dezvoltată. Aceste cerințe au fost stabilite având în vedere utilizarea sistemului într-o locuință obișnuită, unde utilizatorul final trebuie să poată controla dispozitivele într-un mod simplu și intuitiv.

Prima cerință este controlul iluminatului. Utilizatorul trebuie să poată porni și opri luminile din orice cameră folosind un întrerupător fizic sau aplicația software. În plus, sistemul trebuie să permită reglarea intensității luminii pentru surse compatibile cu control PWM [2].

O a doua cerință este controlul dispozitivelor care utilizează telecomandă infraroșu, cum ar fi televizoarele sau aparatele de aer condiționat. Sistemul trebuie să permită învățarea codurilor IR [7] și retransmiterea acestora la comandă, astfel încât dispozitivele existente să poată fi integrate fără modificări.

O altă cerință importantă este monitorizarea mediului. Sistemul trebuie să permită conectarea unor senzori de temperatură și umiditate [15], iar valorile măsurate trebuie să poată fi vizualizate în aplicație și utilizate în scenarii de automatizare.

De asemenea, este necesară stabilitatea comunicației. Rețeaua trebuie să funcționeze chiar dacă unele noduri nu sunt în raza directă de comunicare, iar pierderile de mesaje trebuie reduse [10]. Din acest motiv a fost ales protocolul Bluetooth Mesh [3], care permite retransmiterea mesajelor între noduri.

Sistemul trebuie să fie scalabil, astfel încât să poată fi adăugate noi noduri fără modificarea firmware-ului existent [3]. Utilizatorul trebuie să poată introduce un dispozitiv nou în rețea prin procesul de provizionare și configurare.

O altă cerință este existența unei interfețe grafice de control [13]. Aplicația trebuie să permită vizualizarea nodurilor, trimiterea de comenzi și definirea unor scenarii de automatizare. Scenariile trebuie să permită executarea mai multor acțiuni simultan, de exemplu aprinderea luminii și transmiterea unei comenzi IR.

În plus, aplicația trebuie să comunice cu gateway-ul folosind un protocol standard [9], pentru a permite extinderea sistemului. În proiectul realizat, comunicația dintre aplicație și gateway se face prin MQTT [9], utilizând un broker Mosquitto [14] și două topicuri principale pentru trimiterea și recepția mesajelor.

3.3. Prezentarea generală a sistemului

Sistemul realizat este alcătuit din mai multe noduri inteligente interconectate printr-o rețea Bluetooth Mesh [3]. Fiecare nod este bazat pe microcontrolerul ESP32-C3 [2] și îndeplinește un rol specific, în funcție de funcționalitatea implementată.

Tipurile de noduri dezvoltate sunt următoarele:

- nod întrerupător
- nod pentru controlul iluminatului
- nod IR pentru controlul dispozitivelor externe
- nod senzor de mediu
- nod gateway

Toate nodurile rulează firmware dezvoltat în ESP-IDF [5] și utilizează stiva Bluetooth Mesh pentru comunicare [3]. Schimbul de mesaje se realizează pe baza modelului publish–subscribe [3], ceea ce permite transmiterea comenzilor fără cunoașterea explicită a destinatarului. Nodurile interesate recepționează mesajele în funcție de adresele la care sunt abonate.

Gateway-ul reprezintă elementul central de interconectare între rețeaua mesh și aplicația software. Acesta funcționează în modul Proxy [3] și permite comunicarea dintre dispozitivele Bluetooth Mesh și infrastructura IP. Prin intermediul gateway-ului, comenzile generate în aplicație sunt transmise către nodurile din rețea, iar datele colectate de acestea sunt retransmise către interfața de control.

În implementarea finală, aplicația de control și serviciile de comunicație nu rulează pe calculatorul utilizatorului, ci pe o platformă embedded dedicată, pentru a asigura funcționarea autonomă a sistemului. În acest scop a fost utilizată placa Raspberry Pi Zero 2 W [16], pe care rulează sistemul de operare Linux, aplicația de control și brokerul MQTT [14].

Comunicarea dintre aplicație și gateway se realizează prin protocolul MQTT [9], utilizând două topicuri principale:

- mesh/in – pentru comenzile transmise din aplicație către rețeaua Mesh
- mesh/out – pentru mesajele primite din rețea și retransmise către aplicație

Această arhitectură permite controlul centralizat al tuturor nodurilor și monitorizarea stării sistemului în timp real.

Un avantaj important al soluției propuse este suportul pentru scenarii de automatizare. Acestea sunt definite la nivelul aplicației Flask [13] și permit executarea mai multor acțiuni în mod coordonat. Scenariile sunt salvate într-un fișier de configurare (de tip JSON) și sunt interpretate de aplicație, care

transmite comenzile necesare către noduri prin gateway [9]. Astfel, se pot realiza automatizări precum controlul simultan al iluminatului, trimiterea de comenzi IR sau reglarea parametrilor în funcție de anumite condiții.

În continuare vor fi prezentate în detaliu arhitectura sistemului, platforma hardware și implementarea fiecărui tip de nod.

3.4. Arhitectura sistemului

Arhitectura sistemului dezvoltat este una distribuită, bazată pe protocolul Bluetooth Mesh [3], în care fiecare dispozitiv reprezintă un nod independent capabil să transmită, să primească și, în anumite cazuri, să retransmită mesaje. Spre deosebire de sistemele clasice bazate pe o topologie de tip stea, unde toate dispozitivele comunică printr-un punct central, în rețeaua mesh fiecare nod poate contribui la propagarea mesajelor [4], ceea ce permite extinderea acoperirii și creșterea fiabilității.

Structura generală a sistemului este formată din trei componente principale:

- rețeaua Bluetooth Mesh formată din noduri ESP32-C3 [2]
- gateway-ul care face legătura între mesh și aplicația software
- aplicația de control realizată în Python/Flask [13] care comunică prin MQTT [9]

În Figura 3.1 este prezentată arhitectura generală și interconectarea nodurilor în cadrul sistemului propus.

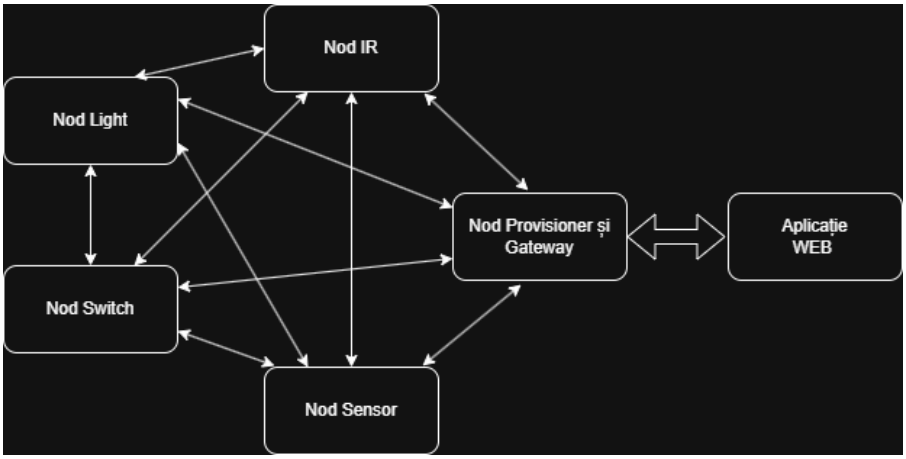


Figura 3.1. Schema logică a rețelei

În această arhitectură, nodurile sunt conectate logic într-o rețea mesh, iar gateway-ul permite accesul la rețea din exterior. Aplicația software nu comunică direct prin Bluetooth Mesh, ci prin intermediul gateway-ului, folosind protocolul MQTT [9].

3.4.1. Comunicarea în rețeaua Bluetooth Mesh

Protocolul Bluetooth Mesh utilizează un mecanism de transmitere bazat pe flooding controlat [10]. Atunci când un nod trimite un mesaj, acesta este recepționat de nodurile din apropiere, care pot retransmite mesajul dacă sunt configurate cu rolul de Relay [3]. Pentru a evita propagarea infinită a mesajelor, fiecare pachet conține un parametru TTL (Time To Live), care indică numărul maxim de retransmisii permise [3, 10].

În sistemul realizat, unele noduri sunt configurate ca Relay pentru a permite comunicarea între camere diferite sau între noduri aflate la distanță. Acest lucru este important în aplicațiile reale, unde nu toate dispozitivele se află în raza directă de acțiune [4].

Mecanismul de retransmitere multi-hop din cadrul rețelei și modul în care mesajele sunt propagate prin nodurile de tip Relay sunt ilustrate în Figura 3.2.



Figura 3.2. Diagramă Mesh Relay

Bluetooth Mesh definește mai multe roluri pentru noduri [3]:

– Relay – retransmite mesajele altor noduri
– Proxy – permite accesul din exterior prin GATT (Generic Attribute Profile)
– Friend – stochează mesaje pentru noduri low power
– Low Power – nod cu consum redus care nu ascultă permanent

În sistemul realizat, gateway-ul este configurat ca Proxy [3], iar nodurile alimentate permanent pot avea rol de Relay.

3.4.2. Modelul publish–subscribe

Comunicarea între nodurile din rețeaua Bluetooth Mesh nu se realizează prin conexiuni directe între două dispozitive, ci prin intermediul modelului publish–subscribe [3]. În acest model, un nod nu transmite mesaje către un destinatar specific, ci publică mesaje către o anumită adresă, iar nodurile interesate de acea adresă le recepționează și le procesează.

Mesajele pot fi trimise către [3]:

– adrese unicast (un singur nod)
– adrese de grup (mai multe noduri simultan)
– adrese virtuale (definite pe baza unui identificator logic)

Fiecare nod este configurat să se aboneze (subscribe) la una sau mai multe adrese, în funcție de rolul său în rețea. De exemplu, nodurile pentru controlul iluminatului pot fi abonate la o adresă de grup comună, astfel încât să reacționeze simultan la aceeași comandă.

Publicarea (publish) mesajelor se face către o adresă configurată în prealabil, de regulă în timpul procesului de configurare realizat prin interfața software. Această abordare permite o flexibilitate ridicată, deoarece relațiile dintre noduri pot fi modificate fără a schimba firmware-ul.

Un avantaj important al modelului publish–subscribe este decuplarea dintre emițător și receptor [3]. Nodurile nu trebuie să cunoască identitatea celorlalte dispozitive, ci doar adresele utilizate pentru comunicare. Acest lucru permite scalarea facilă a rețelei și adăugarea de noi dispozitive fără modificări majore [6].

Modelul arhitectural bazat pe adrese de grup și decuplarea nodurilor prin intermediul paradigmei publish-subscribe sunt exemplificate în Figura 3.3.

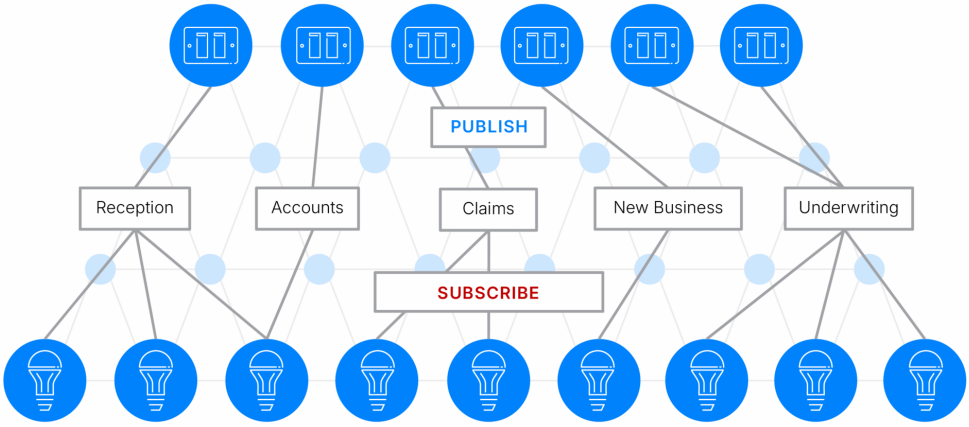


Figura 3.3. Modelul Publish-Subscribe[1]

3.4.3. Arhitectura gateway-ului

Gateway-ul reprezintă componenta care face legătura între rețeaua Bluetooth Mesh și aplicația software de control. Acesta este realizat folosind un microcontroler ESP32-C3 [2], care permite utilizarea simultană a interfețelor Bluetooth Low Energy și Wi-Fi.

Rolul gateway-ului este de a primi comenzi din aplicația software prin protocolul MQTT [9], de a le interpreta și de a le transmite în rețeaua Mesh sub forma mesajelor corespunzătoare modelelor Bluetooth Mesh [6]. De asemenea, gateway-ul recepționează mesaje din rețea și le transmite către aplicația software.

Comunicarea dintre aplicație și gateway se realizează folosind mesaje în format JSON, pentru a permite transmiterea mai multor parametri într-un mod standardizat și ușor de extins.

```
1  {
2    "cmd": "ON",
3    "addr": "0x0005",
4    "val": 1
5  }
```

Listing 3.1. Exemplu de mesaj JSON pentru comanda de acționare a unui nod

```
1  {
2    "type": "data",
3    "addr": "0x0005",
4    "key": "STATE",
5    "val": 1
6  }
```

Listing 3.2. Exemplu de mesaj JSON pentru transmiterea stării unui nod către aplicație

Gateway-ul procesează mesajele folosind o bibliotecă JSON [17] și decide tipul de comandă care trebuie transmis în rețeaua Mesh, în funcție de câmpurile `cmd`, `addr` și `val`.

Pentru comunicarea MQTT [9] au fost definite două topicuri principale: `mesh/in`, utilizat pentru comenzile trimise din aplicație către gateway, și `mesh/out`, folosit pentru mesajele primite din rețea și retransmise către aplicație.

Această arhitectură permite separarea completă între aplicația software și firmware-ul nodurilor, ceea ce face sistemul ușor de extins și modificat fără recompilarea codului pentru dispozitive.

[1]Imagine preluată din ghidul oficial Bluetooth SIG https://www.bluetooth.com/bluetooth-mesh-networking-primer/

3.4.4. Scenarii de automatizare

Un element important al sistemului este posibilitatea de a defini scenarii de automatizare din interfața aplicației. Un scenariu reprezintă o secvență de acțiuni care sunt executate simultan sau secvențial atunci când apare un eveniment. Exemple de scenarii implementate:

– pornirea și oprirea aerului condiționat în funcție de temperatura din cameră
– pornirea televizorului la stingerea luminii
– scenarii bazate pe timp (timer), executate automat la intervale prestabilite

Scenariile sunt definite în aplicația Flask [13] și sunt salvate într-un fișier JSON. Când utilizatorul activează un scenariu, aplicația trimite mai multe mesaje MQTT [9] către gateway, care sunt apoi propagate în rețeaua mesh [3].

Această abordare permite extinderea sistemului fără modificarea firmware-ului nodurilor, deoarece logica de automatizare este implementată la nivelul aplicației.

3.4.5. Avantajele arhitecturii alese

Arhitectura bazată pe Bluetooth Mesh [3] și gateway MQTT [9] oferă mai multe avantaje:

– nu depinde de cloud
– permite control local
– este scalabilă [3]
– suportă multe noduri [4]
– permite automatizări complexe [1]
– consum redus de energie [4]
– securitate prin criptare AES-CCM [3]

Dezavantajele principale sunt:

– implementare mai complexă [5]
– necesitatea configurării provizionării [3]
– limitări de viteză comparativ cu Wi-Fi [8]
– necesitatea unui gateway pentru control extern

În ciuda acestor limitări, arhitectura aleasă este potrivită pentru aplicații domotice, deoarece oferă un echilibru bun între flexibilitate, cost și performanță [1].

3.4.6. Procesul de provizionare și configurare a nodurilor

Provizionarea reprezintă mecanismul prin care un dispozitiv neprovizionat (unprovisioned device) este admis în rețeaua Bluetooth Mesh, primind identitate criptografică și parametrii necesari pentru comunicare [3]. În sistemul realizat, gateway-ul îndeplinește rolul de Provizioner — singura entitate autorizată să admită noduri noi în rețea.

Procesul complet se desfășoară în două etape distincte: provizionarea propriu-zisă (atribuirea adresei și a cheilor de rețea) și configurarea post-provizionare (legarea cheilor de aplicație la modele și setarea parametrilor de publicare). Separarea celor două etape este impusă de specificația Bluetooth Mesh [3] și reflectată direct în implementare.

Descoperirea dispozitivelor neprovizionate:

Înainte de provizionare, gateway-ul scanează continuu canalele BLE pentru pachete de tip Unprovisioned Device Beacon. Fiecare dispozitiv neprovizionat transmite periodic un astfel de beacon care conține UUID-ul său unic de 16 octeți. În implementarea realizată, gateway-ul filtrează dispozitivele pe baza primilor doi octeți ai UUID-ului (**0xDD**, **0xDD**), ignorând dispozitivele BLE din jur care nu aparțin sistemului:

```
esp_ble_mesh_provisioner_set_dev_uuid_match(
    (uint8_t[]){0xdd, 0xdd}, 2, 0x00, false
);
```

Listing 3.3. Filtrare UUID

Dispozitivele detectate sunt stocate temporar într-o listă de scanare (**scan_results**) și raportate aplicației Flask prin MQTT ca mesaje de tip **unprov**, conținând UUID-ul și puterea semnalului (RSSI). Lista se curăță automat din Python după 10 secunde dacă dispozitivul nu mai este vizibil.

Sistemul suportă două moduri de operare, selectabile din interfața grafică: modul **MANUAL**, în care utilizatorul selectează explicit dispozitivul din lista de scanare și apasă butonul „Add", și modul **AUTO**, în care gateway-ul provizionează automat orice dispozitiv compatibil detectat, fără intervenție umană.

Provizionarea propriu-zisă:

La inițierea provizionării (manual sau automat), gateway-ul apelează **esp_ble_mesh_provisioner_add_unprov_dev** cu flagurile **ADD_DEV_START_PROV_NOW_FLAG** și **ADD_DEV_RM_AFTER_PROV_FLAG**. Stiva Bluetooth Mesh execută protocolul de provizionare conform specificației [3], care include:

- schimbul de capabilități între Provizioner și dispozitiv
- generarea și schimbul de chei criptografice, folosind algoritmul ECDH (Elliptic-Curve Diffie–Hellman)
- distribuirea Network Key (NetKey) și atribuirea adresei unicast

La finalizarea cu succes, callback-ul **ESP_BLE_MESH_PROVISIONER_PROV_COMPLETE_EVT** este apelat, furnizând adresa unicast atribuită nodului. Adresele sunt alocate secvențial începând de la **0x0005**, gateway-ul ocupând adresa fixă **0x0001**. Nodul este înregistrat în baza de date internă și este declanșată imediat cascada de configurare.

Cascada de configurare post-provizionare:

După provizionare, nodul deține doar NetKey-ul și adresa unicast, dar nu poate încă procesa comenzi de aplicație — modelele sale nu sunt legate de nicio cheie de aplicație. Configurarea se realizează printr-o secvență de mesaje trimise prin modelul Configuration Client al gateway-ului, implementată ca o mașină de stări cu 7 pași:

Pasul 0 — Composition Data Get: Gateway-ul solicită datele de compoziție ale nodului, care descriu structura sa internă: elementele, modelele SIG standard și modelele vendor implementate. Răspunsul este analizat pentru a determina capabilitățile nodului (**has_onoff**, **has_level**, **has_sensor**, **has_ir**, **has_onoff_client**). Aceste informații sunt publicate prin MQTT ca mesaj de tip **discovery** și salvate în baza de date NVS a gateway-ului.

Pasul 0.5 — App Key Add: Imediat după analiza compoziției, gateway-ul distribuie Application Key (AppKey) către nod prin mesajul **APP_KEY_ADD**. AppKey-ul este o cheie de 16 octeți care permite criptarea la nivel de aplicație, separată de criptarea la nivel de rețea asigurată de NetKey [3].

Pașii 1–5 — Model App Bind: Pentru fiecare model prezent în nod, gateway-ul trimite un mesaj **MODEL_APP_BIND** care leagă AppKey-ul la modelul respectiv. Ordinea este: Generic OnOff Server (pasul 1), Generic Level Server (pasul 2), Sensor Server (pasul 3), Vendor IR Server (pasul 4), Generic OnOff Client (pasul 5). Pașii corespunzători modelelor absente sunt săriți automat.

Pasul 6 — Model Publication Set: Pentru nodurile cu Generic OnOff Server, gateway-ul configurează adresa de publicare a modelului către propria adresă (**0x0001**), permițând nodului să raporteze schimbările de stare către gateway fără solicitare explicită.

Fiecare pas este executat cu un delay de 200 ms față de confirmarea pasului anterior, pentru a evita congestia stivei Mesh. Un watchdog task verifică la fiecare 3 secunde dacă vreun nod a depășit

15 secunde în configurare fără confirmare, caz în care configurarea este abandonată și nodul este eliminat din baza de date.

La finalizarea tuturor pașilor, gateway-ul publică evenimentul **CONFIG_DONE** prin MQTT, iar aplicația Flask actualizează interfața grafică cu nodul nou configurat, gata de utilizare.

Persistența configurației:

Baza de date a nodurilor (**node_db**) este salvată în `NVS` după fiecare modificare și reîncărcată la repornirea gateway-ului. Aceasta stochează adresa, capabilitățile și starea de configurare a fiecărui nod. Rețeaua Mesh în sine (NetKey, adrese, chei) este persistată automat de stiva ESP-IDF în propriul său spațiu `NVS`. Astfel, după o repornire a gateway-ului, nodurile existente nu necesită re-provizionare, ele sunt recunoscute automat și pot fi controlate imediat.

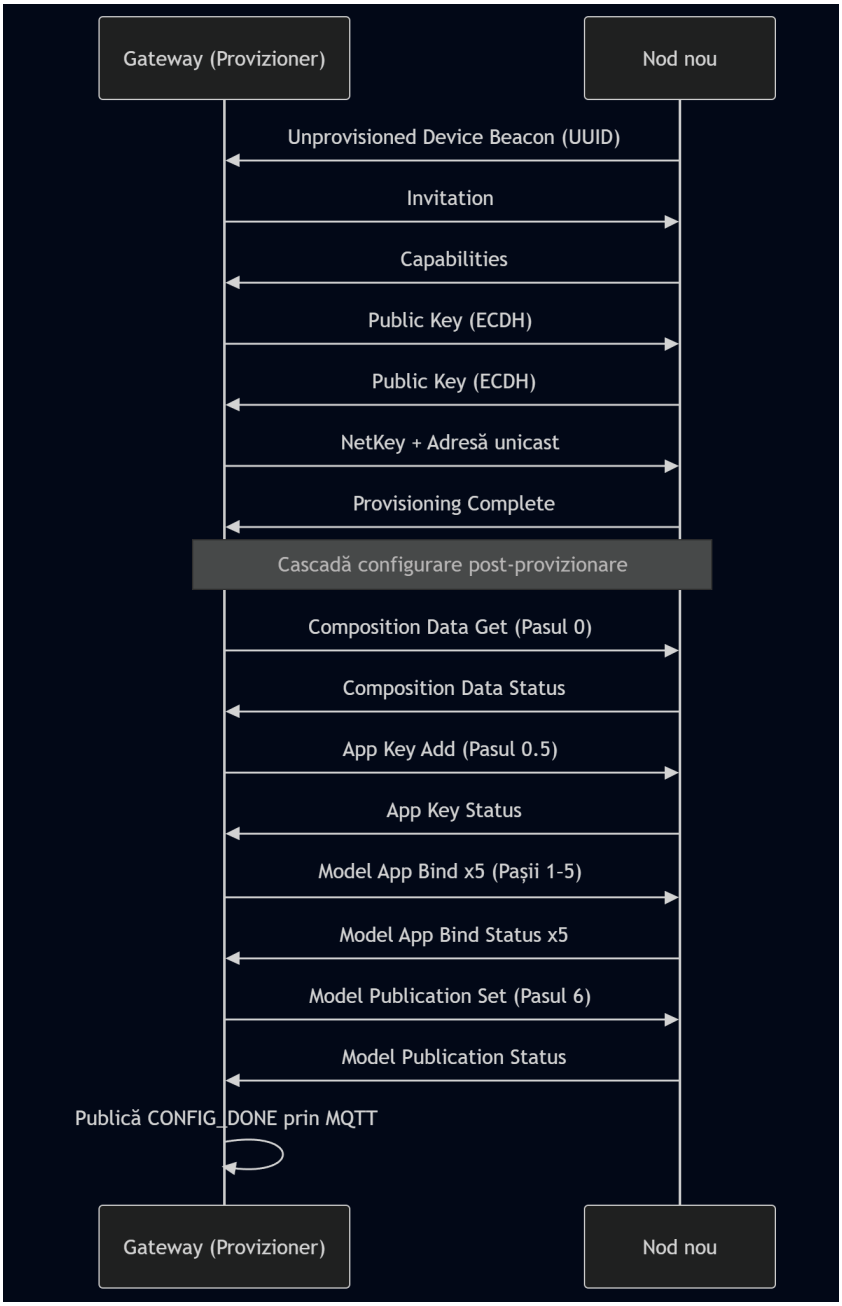


Figura 3.4. Diagrama secvențială a procesului de provizionare și configurare a nodurilor

3.5. Platforma hardware

Alegerea platformei hardware a avut un rol esențial în realizarea sistemului, deoarece toate nodurile trebuie să fie compatibile cu protocolul Bluetooth Mesh și să ofere suficiente resurse pentru

implementarea funcțiilor necesare. În cadrul proiectului s-a ales utilizarea microcontrolerului ESP32-C3 [2], datorită suportului integrat pentru Bluetooth Low Energy, performanței ridicate și costului redus.

Toate nodurile din rețea sunt construite în jurul unor plăci de dezvoltare ESP32-C3, la care sunt conectate diferite componente externe în funcție de rolul nodului.

3.5.1. Microcontrolerul ESP32-C3

ESP32-C3 este un microcontroler produs de Espressif Systems [2], proiectat pentru aplicații IoT. Acesta utilizează o arhitectură RISC-V pe 32 de biți [11] și poate funcționa la frecvențe de până la 160 MHz. Microcontrolerul include suport hardware pentru Wi-Fi și Bluetooth Low Energy 5.0, cu suport pentru Bluetooth Mesh [2].

Avantajele utilizării ESP32-C3 sunt:

- suport nativ pentru Bluetooth Mesh [2]
- consum redus de energie [4]
- accelerare hardware pentru criptografie [3]
- număr suficient de pini GPIO [2]
- suport bun în ESP-IDF [5]
- cost redus

ESP32-C3 include periferice precum GPIO, PWM, UART, SPI, I2C, ADC și timer hardware [2]. Aceste periferice permit conectarea diferitelor componente externe, cum ar fi senzori, relee sau LED-uri infraroșu.

3.5.2. Tipuri de plăci utilizate în proiect

În cadrul proiectului au fost utilizate două tipuri de plăci bazate pe microcontrolerul `ESP32-C3` [2]:

- `ESP32-C3 SuperMini`
- `ESP32-C3 Mini DevKit`

Placa `ESP32-C3 SuperMini` a fost utilizată pentru majoritatea nodurilor datorită dimensiunilor reduse și costului mic. Aceasta este potrivită pentru noduri precum întrerupător, control iluminat, IR sau senzor. În timpul dezvoltării s-a observat însă că placa `ESP32-C3 SuperMini` prezintă probleme de stabilitate în modul Wi-Fi, cauzate de proiectarea precară a antenei și a traseelor de pe placă [2]. În anumite situații, conexiunea Wi-Fi nu se stabilea corect sau se întrerupea, ceea ce făcea dificilă utilizarea acestei plăci pentru rolul de gateway.

Din acest motiv, pentru nodul gateway s-a ales utilizarea plăcii `ESP32-C3 Mini DevKit`, care dispune de o proiectare mai robustă și oferă două porturi USB-C, unul pentru alimentare și unul pentru programare și comunicație serială [2].

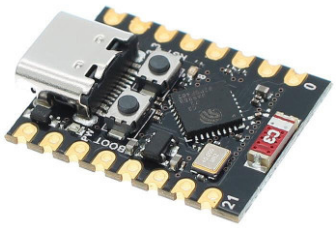


Figura 3.5. Placa de dezvoltare ESP-32-C3 SuperMini[2]



Figura 3.6. Placa de dezvoltare ESP-32-C3 Mini DevKit[3]

Utilizarea acestei plăci a permis o comunicare stabilă cu aplicația software și a eliminat problemele întâmpinate cu SuperMini.

3.5.3. Nodul pentru controlul iluminatului

Acest nod are rolul de a controla un circuit de iluminat, fiind implementat cu un microcontroler `ESP32-C3 SuperMini` [2] și un modul dual MOSFET de putere [18] conectat la un pin GPIO al microcontrolerului. Acesta permite comutarea unui curent mai mare decât ar putea furniza direct pinul microcontrolerului. Modulul este proiectat să funcționeze cu tensiuni cuprinse între 5 VDC și 36 VDC, cu un curent de până la 15A constant, suportând vârfuri de până la 30A [18]. În proiect s-a ales alimentarea unei benzi LED de 12V.

Controlul se realizează printr-un semnal PWM generat de `ESP32-C3` [2], care comandă poarta MOSFET-ului [18] și permite pornirea, oprirea sau reglarea intensității luminii. Schema de conectare este următoarea:

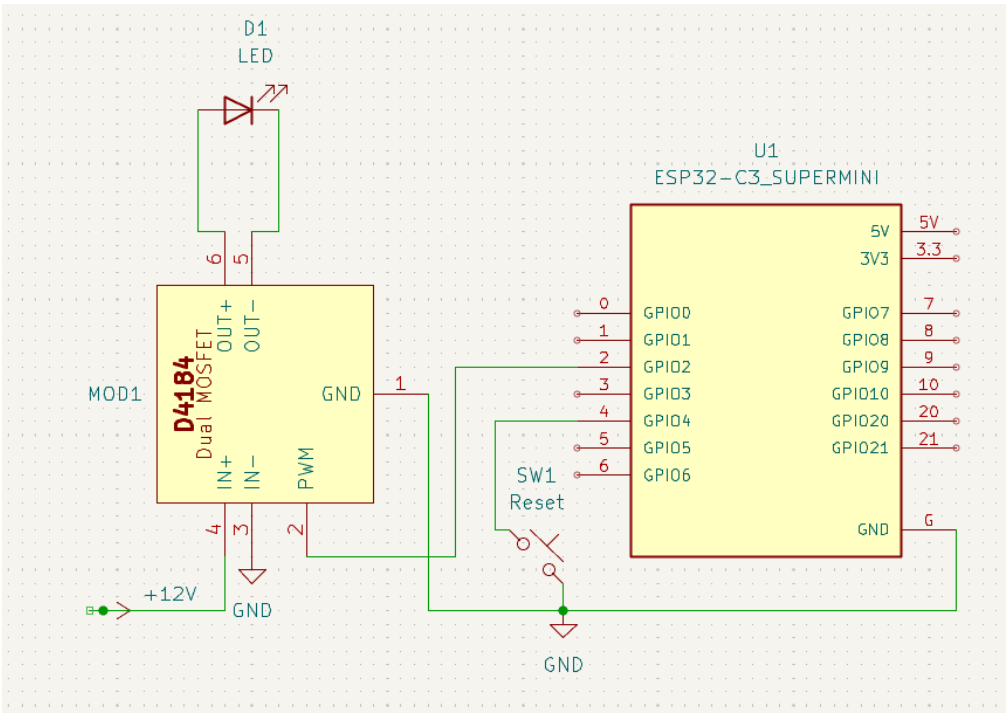


Figura 3.7. Schemă electrică Nod Releu pentru iluminat

Nodul este proiectat pentru a funcționa în rețeaua Bluetooth Mesh [3], primind comenzi de la alte noduri sau de la gateway și controlând ieșirea în funcție de mesajele primite.

[2]Imagine preluată de la https://sigmanortec.ro/placa-dezvoltare-esp32-c3-supermini-33v-wifi-bluetooth

[3]Imagine preluată de la https://www.emag.ro/modul-esp32-c3-devkitm-1-risc-v-wifi-bluetooth-5-0-4mb-flash-24x18x4mm-o-108/pd/DRPGLL3BM/

3.5.4. Nodul întrerupător

Nodul întrerupător este realizat folosind un microcontroler `ESP32-C3 SuperMini` [2] și unul sau mai multe butoane conectate la pini GPIO. Butoanele sunt configurate ca intrări digitale cu rezistențe interne de pull-up, astfel încât în stare normală pinul are nivel logic `HIGH`, iar la apăsare trece în nivel logic `LOW`.

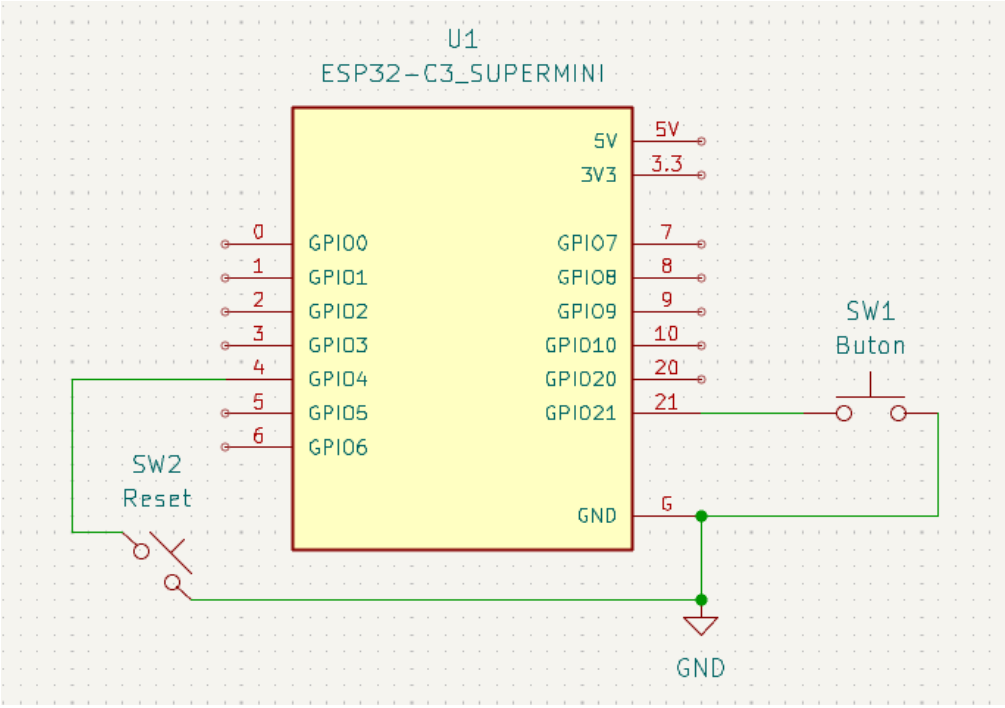


Figura 3.8. Schemă electrică Nod Întrerupător

Detectarea apăsării se realizează prin monitorizarea schimbării stării pinului, evenimentul fiind prelucrat de firmware pentru a genera o acțiune în rețea [5].

Spre deosebire de un întrerupător clasic, acest nod nu controlează direct un consumator, ci funcționează ca un element de comandă în rețeaua Bluetooth Mesh [3]. La apăsarea unui buton, nodul publică un mesaj către o adresă configurată, utilizând modelul `Generic OnOff Client` [6].

Pentru a asigura sincronizarea corectă a stării dispozitivelor, implementarea nu trimite direct o comandă de tip ON/OFF, ci utilizează un mecanism de tip „toggle” bazat pe starea curentă a dispozitivului controlat [6]. Astfel, nodul:

- solicită starea actuală printr-un mesaj `Generic OnOff Get`
- primește răspunsul `Generic OnOff Status`
- determină starea opusă
- transmite comanda `Generic OnOff Set`

Această abordare previne desincronizarea în situațiile în care dispozitivul a fost controlat anterior din aplicație sau de către un alt nod.

Adresa de publicare utilizată pentru transmiterea comenzilor este configurată prin intermediul interfeței software, permițând flexibilitate în definirea relațiilor dintre noduri fără modificarea firmware-ului.

3.5.5. Nodul IR

Nodul IR este utilizat pentru controlul dispozitivelor care folosesc telecomandă cu infraroșu, cum ar fi televizoare, sisteme audio sau aparate de aer condiționat. Spre deosebire de soluțiile care

implementează un protocol IR specific (NEC, Sony, RC5 etc.), abordarea aleasă este capturarea și retransmiterea semnalului brut (RAW), ceea ce conferă nodului compatibilitate universală cu orice dispozitiv controlabil prin infraroșu, indiferent de protocolul utilizat.

Nodul este realizat folosind o placă ESP32-C3 SuperMini [2], un LED infraroșu conectat la GPIO 5 prin intermediul unui tranzistor 2N7000 și o rezistență de 82 Ω, un receptor infraroșu VS1838B [19] conectat la GPIO 3 — cu frecvență purtătoare de 38 kHz, unghi de recepție de ±45° și tensiune de alimentare între 2.5V și 5.5V și trei butoane fizice: GPIO 4 pentru reset, GPIO 9 pentru inițierea procesului de învățare și GPIO 10 pentru retransmiterea semnalului memorat. Raza de acțiune a nodului depinde de intensitatea radiantă a LED-ului, aceasta fiind determinată în principal de curentul de comandă. Deoarece curentul maxim suportat de pinii GPIO ai ESP32-C3 este limitat, pentru a putea utiliza LED-uri de putere, controlul se realizează prin intermediul tranzistorului 2N7000.

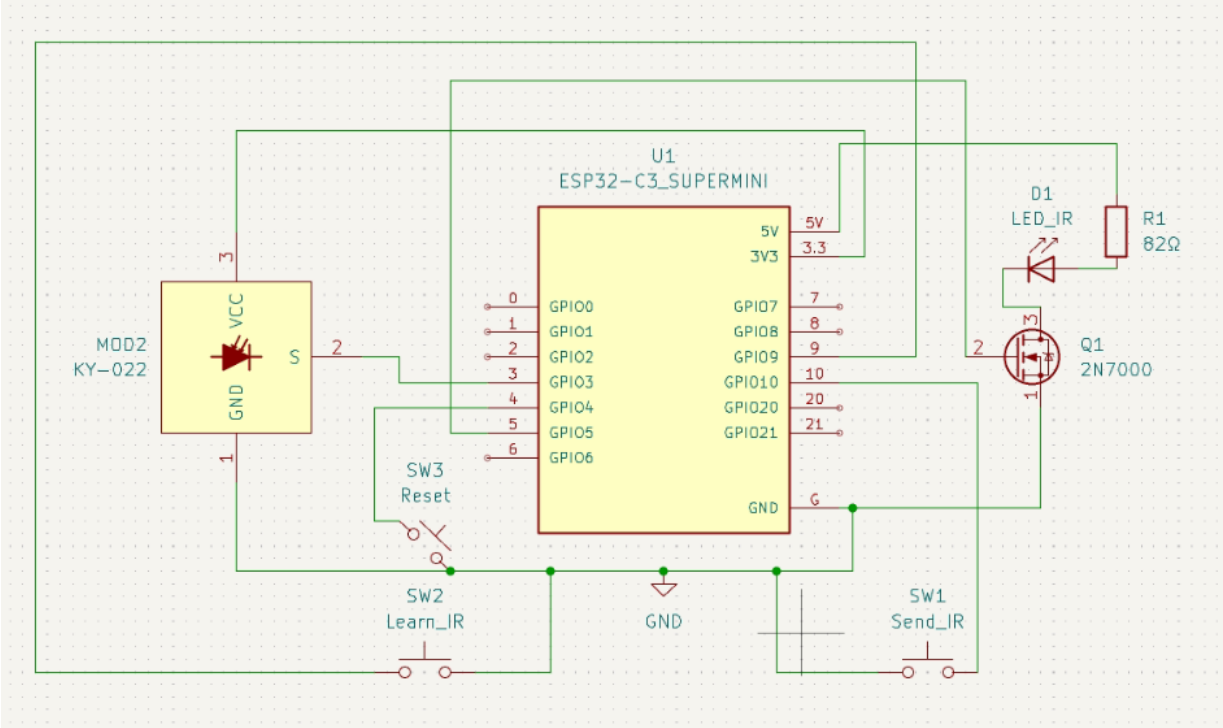


Figura 3.9. Schemă electrică Nod Infraroșu

Generarea și recepția semnalelor infraroșu se realizează folosind perifericul RMT (Remote Control Transceiver) integrat în microcontrolerul ESP32-C3 [7]. RMT este un periferic hardware dedicat care operează independent de procesor, permițând generarea și capturarea precisă a semnalelor modulate fără a solicita CPU-ul în timpul transferului. Rezoluția configurată este de 1 MHz (1 μs per tick), suficientă pentru reprezentarea fidelă a oricărui semnal IR comercial.

Canalul de transmisie aplică hardware o frecvență purtătoare de 38 kHz cu un duty cycle de 33%, valori standard utilizate de majoritatea telecomenzilor comerciale [7]. Canalul de recepție este configurat cu inversarea semnalului activată (`flags.invert_in = true`), deoarece receptorul VS1838B produce un semnal activ-low față de semnalul IR fizic recepționat — fără această inversare, simbolurile capturate ar fi reprezentate cu polaritate incorectă.

Semnalul capturat este stocat în unul din cele 5 sloturi independente definite în memoria nevolatilă NVS a microcontrolerului, permițând controlul mai multor dispozitive sau comenzi diferite de pe același nod. Persistența în NVS asigură că semnalele memorate sunt păstrate după repornirea dispozitivului, fără a fi necesară reluarea procesului de învățare.

Acest nod permite integrarea dispozitivelor clasice controlate prin telecomandă într-un sistem de automatizare bazat pe Bluetooth Mesh [3], fără a fi necesară modificarea hardware a dispozitivelor controlate.

3.5.6. Nodul senzor

Pentru măsurarea temperaturii și umidității relative a fost utilizat senzorul digital `DHT22-(AM2302)` [15], conectat la un pin al microcontrolerului `ESP32-C3` [2]. Senzorul utilizează un protocol de comunicație serial proprietar pe o singură linie de date bidirecțională, cu codificare prin durata impulsului (pulse-width encoding). Comunicarea se desfășoară în regim master-slave, microcontrolerul inițiind fiecare ciclu de citire, iar senzorul răspunzând cu exact 40 de biți de date. Linia de date este menținută în stare `HIGH` prin rezistența externă de pull-up de 10kΩ atât timp cât magistrala este inactivă, valoare adecvată pentru distanțele scurte de cablu utilizate în implementare [15].

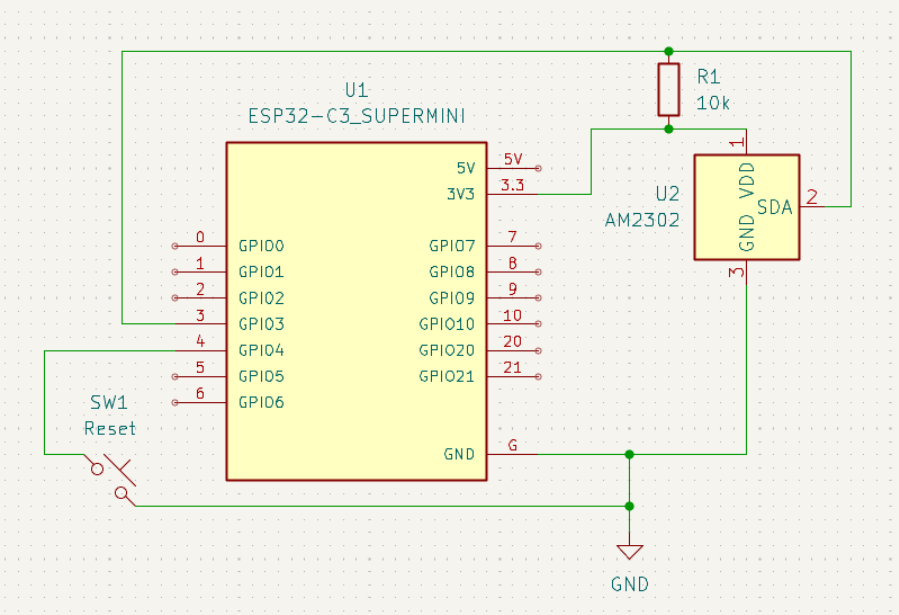


Figura 3.10. Schemă electrică Nod Senzor

Secvența completă de comunicație al protocolului single-bus se desfășoară în trei faze:

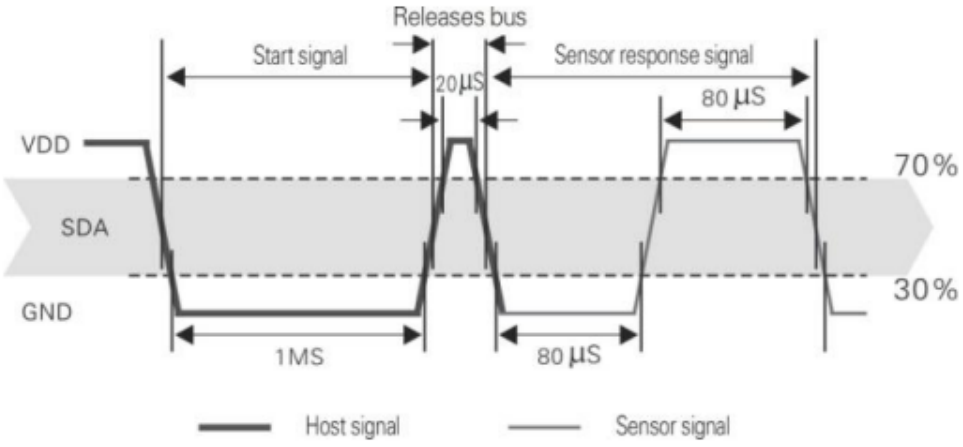


Figura 3.11. Diagrama de timing faza de inițializare și răspuns AM2302 [15]

Faza 1 – Inițializarea (semnalul host): Microcontrolerul configurează pinul de date ca ieșire în regim open-drain și îl trage la `LOW`. Conform specificațiilor producătorului, durata minimă a acestui semnal este de 800 μs, cu valoarea tipică de 1 ms [15]. În implementarea realizată, durata utilizată este de 12 ms, valoare determinată empiric pentru a asigura compatibilitatea cu modulul hardware utilizat - driverul software pe care se bazează implementarea menționează explicit că valori sub 10 ms

nu produc răspuns pe unele variante de hardware. După expirarea acestui interval, microcontrolerul eliberează linia prin setarea nivelului `HIGH` în regim open-drain, lăsând rezistența de pull-up să readucă magistrala la nivel logic 1. Senzorul detectează tranziția și iese din modul standby.

Faza 2 – Răspunsul senzorului: La detectarea semnalului de inițializare, senzorul confirmă prezența sa pe magistrală trăgând linia la `LOW` timp de 80 μs (T_{rel} = 75–85 μs conform datasheet), după care o eliberează la `HIGH` pentru încă 80 μs (T_{reh} = 75–85 μs). Microcontrolerul monitorizează aceste tranziții pentru a confirma că senzorul a răspuns corect înainte de a începe recepția datelor.

Faza 3 – Transmiterea datelor: Senzorul transmite 40 de biți serial, MSB primul. Fiecare bit începe cu o perioadă `LOW` de 50 μs (T_{LOW} = 48–55 μs), reprezentând semnalul de start al bitului. Valoarea bitului este codificată în durata perioadei `HIGH` care urmează, conform Figurii 3.12 [15]:

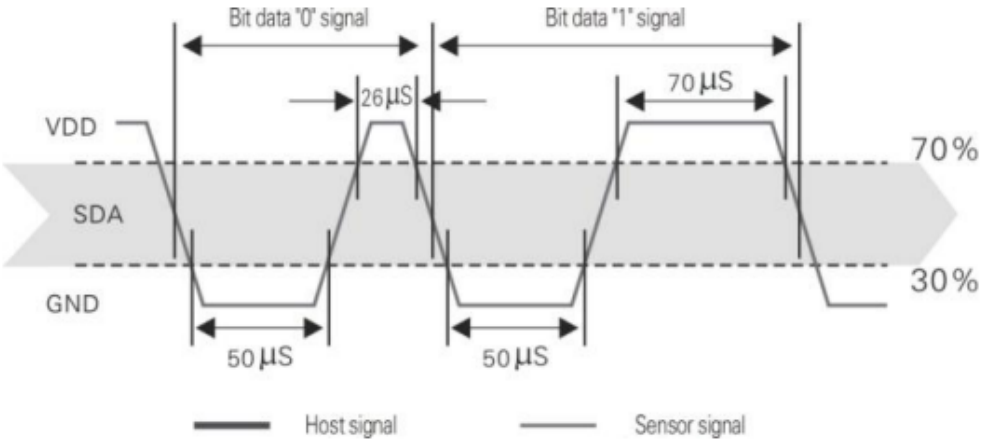


Figura 3.12. Codificarea biților 0 și 1 [15]

– **Bit 0**: perioada `HIGH` are durata de 26–28 μs (T_{H0} = 22–30 μs)
– **Bit 1**: perioada `HIGH` are durata de 70 μs (T_{H1} = 68–75 μs)

Parametrii compleți de timing conform documentației oficiale a producătorului [15] sunt prezentați în Figura 3.13.

Symbol	Parameter	min	typ	max	Unit
T_{be}	Host the start signal down time	0.8	1	20	mS
T_{go}	Bus master has released time	20	30	200	μS
T_{rel}	Response to low time	75	80	85	μS
T_{reh}	In response to high time	75	80	85	μS
T_{LOW}	Signal "0", "1" low time	48	50	55	μS
T_{H0}	Signal "0" high time	22	26	30	μS
T_{H1}	Signal "1" high time	68	70	75	μS
T_{en}	Sensor to release the bus time	45	50	55	μS

Figura 3.13. Parametrii de timing ai protocolului single-bus AM2302 [15]

Structura cadrului de 40 de biți:

Cei 40 de biți sunt structurați după cum urmează: primii 16 biți conțin valoarea umidității relative, urmați de 16 biți pentru temperatură, și 8 biți de checksum [15]. Valorile sunt transmise ca

numere întregi reprezentând de 10 ori valoarea reală — de exemplu, o temperatură de 35.1°C este transmisă ca valoarea 351, iar o umiditate de 65.2%RH ca valoarea 652. Bitul cel mai semnificativ al cuvântului de temperatură (`bit 15`) are semnificație de semn: valoarea 1 indică temperatură negativă. Checksum-ul se calculează ca suma celor patru octeți de date, reținând doar cei mai puțin semnificativi 8 biți (`mod 256`). Dacă valoarea calculată de microcontroler nu coincide cu checksum-ul primit, citirea este invalidă și trebuie repetată.

Implementarea driverului:

Protocolul DHT22 impune constrângeri stricte de timing, deoarece diferența dintre un bit 0 (26 μs `HIGH`) și un bit 1 (70 μs `HIGH`) este de ordinul zecilor de microsecunde. Orice întrerupere a execuției firmware-ului în intervalul de recepție poate duce la eșecul citirii.

Driverul utilizat rezolvă această problemă prin înregistrarea timpului absolut al fiecărui front descrescător (falling edge) pe linia de date, folosind întreruperi GPIO și timerul hardware de înaltă rezoluție al ESP32-C3 (`esp_timer_get_time()`). În loc să măsoare direct durata perioadei `HIGH` a fiecărui bit — abordare vulnerabilă la întreruperi de execuție — driverul măsoară intervalul de timp dintre două fronturi descrescătoare consecutive. Această valoare înglobează atât perioada `LOW` (50 μs, constantă) cât și perioada `HIGH` (variabilă), rezultând:

- **bit 0**: ∼76–93 μs între două fronturi descrescătoare (*falling edges*)
- **bit 1**: ∼120–135 μs între două fronturi descrescătoare (*falling edges*)

Pragul de decizie aplicat în implementare este de 110 μs, ales la mijlocul intervalului dintre cele două valori posibile, ceea ce oferă o marjă de zgomot mai mare față de abordarea directă bazată pe 50 μs.

Citirea se declanșează periodic dintr-un task FreeRTOS la intervale de 10 secunde, respectând cu o marjă confortabilă cerința din specificații de minimum 2 secunde între citiri consecutive [15].

Valorile citite sunt convertite în format numeric și transmise în rețeaua Bluetooth Mesh folosind modelul standard `Sensor Server` [6], la cererea gateway-ului, pentru a evita traficul inutil în rețea.

3.5.7. Alimentarea nodurilor

Majoritatea nodurilor din sistem sunt alimentate la o tensiune de 5V prin intermediul portului USB. Plăcile `ESP32-C3` utilizate includ un regulator intern care convertește această tensiune la 3.3V, necesară funcționării microcontrolerului și perifericelor asociate [2].

În cadrul implementării prezentate, alimentarea s-a realizat prin cablu USB, ceea ce permite o configurare rapidă și stabilă a nodurilor. Pentru utilizarea în aplicații reale, pot fi luate în considerare mai multe soluții de alimentare, în funcție de tipul nodului și de amplasarea acestuia:

- alimentatoare USB dedicate
- baterii Li-Ion sau Li-Po, pentru noduri independente
- surse de 5V integrate în instalația electrică

Consumul redus al protocolului Bluetooth Mesh [4], împreună cu posibilitatea utilizării modurilor de economisire a energiei (`Friend/Low Power`) [3], permite alimentarea pe baterie pentru anumite tipuri de noduri, în special pentru cele care nu necesită funcționare continuă (de exemplu, senzori sau întrerupătoare).

În cazul nodurilor de tip releu, care controlează sarcini de putere (cum ar fi benzi LED), este necesară utilizarea unei surse externe adecvate pentru alimentarea consumatorului, separată de alimentarea microcontrolerului [18].

3.5.8. Avantaje și dezavantaje ale platformei hardware

Alegerea platformei hardware a fost realizată astfel încât să se obțină un compromis optim între cost, performanță, consum de energie și suport pentru tehnologiile necesare implementării rețelei

Bluetooth Mesh [3]. În cadrul proiectului au fost utilizate plăci bazate pe microcontrolerul ESP32-C3 [2], precum și o placă Raspberry Pi Zero 2 W pentru rularea aplicației de control și a serviciilor software.

Unul dintre principalele avantaje ale utilizării microcontrolerului ESP32-C3 este suportul hardware nativ pentru Bluetooth Low Energy și Bluetooth Mesh, ceea ce permite implementarea unei rețele distribuite fără a fi necesare module externe suplimentare [2]. Acest lucru simplifică proiectarea și reduce costul total al sistemului.

Un alt avantaj important este consumul redus de energie comparativ cu soluțiile bazate exclusiv pe Wi-Fi [8]. Deoarece multe noduri pot fi alimentate permanent, iar altele pot funcționa în moduri low-power [3], utilizarea BLE Mesh permite obținerea unei eficiențe energetice mai bune decât în cazul comunicației Wi-Fi directe între dispozitive.

Plăcile ESP32-C3 oferă un număr suficient de pini GPIO pentru conectarea senzorilor, a modulelor de control al iluminatului [18] și a altor periferice, fără a fi necesare circuite suplimentare complexe [2]. În plus, suportul oferit de mediul ESP-IDF [5] și documentația disponibilă facilitează dezvoltarea firmware-ului și integrarea funcțiilor Mesh.

Utilizarea plăcii Raspberry Pi Zero 2 W [16] pentru rularea aplicației și a brokerului MQTT Mosquitto [14] reprezintă de asemenea un avantaj, deoarece permite separarea logicii de control de nodurile din rețea. În acest mod, sistemul poate funcționa autonom, fără a depinde de un calculator permanent conectat, iar serviciile software pot rula continuu pe un dispozitiv dedicat.

Costul redus al componentelor utilizate reprezintă un alt avantaj important. Atât plăcile ESP32-C3, cât și Raspberry Pi Zero 2 W sunt accesibile și ușor de găsit, ceea ce face ca soluția propusă să poată fi replicată și extinsă fără investiții mari.

Un dezavantaj observat în timpul dezvoltării este legat de variațiile existente între diferite plăci bazate pe ESP32-C3 [2]. Unele versiuni compacte, precum ESP32-C3 SuperMini, au prezentat probleme de stabilitate în funcționarea modulului Wi-Fi, cauzate de proiectarea hardware a plăcii. Din acest motiv, pentru nodul gateway a fost necesară utilizarea unei plăci ESP32-C3 DevKit mai robuste, care oferă o antenă mai bine proiectată.

Un alt dezavantaj al utilizării Bluetooth Mesh este debitul redus de date comparativ cu alte tehnologii de comunicație, precum Wi-Fi [8]. Protocolul Mesh este optimizat pentru mesaje scurte și frecvente, dar nu este potrivit pentru transferuri mari de date. În cadrul proiectului acest lucru nu reprezintă o problemă majoră, deoarece sistemul transmite doar comenzi și valori de senzori, însă limitează utilizarea pentru aplicații mai complexe.

Configurarea inițială a rețelei Mesh este mai complexă decât în cazul unei rețele clasice Wi-Fi [5]. Procesul de provizionare, configurarea adreselor și gestionarea cheilor de securitate necesită pași suplimentari și o bună înțelegere a protocolului [3].

Utilizarea unei plăci Raspberry Pi pentru rularea aplicației introduce o componentă suplimentară în sistem, ceea ce crește ușor complexitatea generală. Totuși, acest dezavantaj este compensat de flexibilitatea ridicată și de posibilitatea extinderii ulterioare a sistemului.

Un alt aspect care trebuie luat în considerare este faptul că nodurile alimentate la baterie trebuie configurate cu atenție pentru a reduce consumul de energie, deoarece Bluetooth Mesh implică retransmiterea mesajelor, ceea ce poate crește consumul dacă nu sunt utilizate corect rolurile Low Power și Friend [3].

3.6. Tehnologii software utilizate

Implementarea sistemului a necesitat utilizarea mai multor tehnologii software, atât pe partea de firmware pentru nodurile ESP32-C3, cât și pe partea de aplicație pentru controlul centralizat. Alegerea tehnologiilor a fost făcută astfel încât sistemul să fie stabil, extensibil și compatibil cu protocolul Bluetooth Mesh [3].

Principalele componente software utilizate sunt:

– ESP-IDF [5] pentru dezvoltarea firmware-ului

- stiva Bluetooth Mesh integrată în ESP-IDF [5]
- limbajul C/C++ pentru noduri
- Python și Flask [13] pentru aplicație
- MQTT [9] pentru comunicația cu gateway-ul
- Mosquitto [14] ca broker MQTT
- împachetarea comenzilor din MQTT în format JSON

Prin combinarea acestor tehnologii s-a obținut o arhitectură flexibilă, în care nodurile funcționează independent, iar logica de control poate fi implementată în aplicația software.

3.6.1. Mediul de dezvoltare ESP-IDF

Firmware-ul pentru nodurile bazate pe ESP32-C3 a fost dezvoltat utilizând ESP-IDF (Espressif IoT Development Framework) [5], mediul oficial de dezvoltare oferit de producător. Acesta pune la dispoziție un set complet de instrumente și biblioteci necesare dezvoltării aplicațiilor embedded pentru familia ESP32.

ESP-IDF oferă suport pentru [5]:

- drivere hardware pentru periferice (GPIO, PWM, UART etc.) [2]
- sistem de operare în timp real bazat pe FreeRTOS [12]
- comunicații Wi-Fi și Bluetooth
- biblioteci dedicate pentru implementarea protocolului Bluetooth Mesh [3]
- sistem de build bazat pe CMake și toolchain dedicat

Utilizarea ESP-IDF permite un control detaliat asupra resurselor hardware și asupra modului de execuție al aplicației, oferind o flexibilitate și performanță superioare comparativ cu medii de dezvoltare simplificate, cum ar fi Arduino [5].

Structura unui proiect ESP-IDF este modulară și organizată pe componente [5]. În mod tipic, aceasta include:

- fișierul principal de aplicație (main.c / main.cpp)
- fișiere de configurare (sdkconfig)
- componente pentru Bluetooth Mesh [3]
- module pentru drivere hardware
- fișiere pentru implementarea modelelor custom [6]

Această organizare modulară facilitează reutilizarea codului și separarea clară între funcționalitățile comune și cele specifice fiecărui tip de nod.

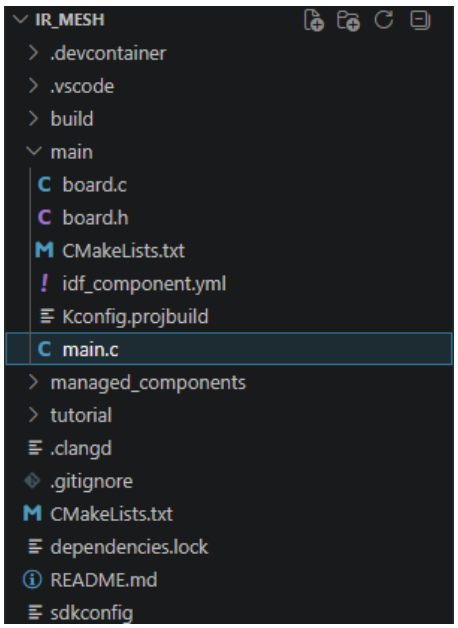


Figura 3.14. Structură Proiect

3.6.2. Stiva Bluetooth Mesh

`ESP-IDF` include o implementare completă a protocolului Bluetooth Mesh [5], conform specificației Mesh Profile 1.0 [3]. Această stivă oferă toate mecanismele necesare pentru realizarea unei rețele distribuite de dispozitive, incluzând:

- provizionarea nodurilor [3]
- configurarea parametrilor de rețea [3]
- gestionarea adreselor și a grupurilor [6]
- criptarea și securizarea comunicației (`AES-CCM`) [3]
- retransmiterea mesajelor (*relay*) [8]
- suport pentru modele standard și modele vendor [6]

Funcționalitatea fiecărui nod este definită prin intermediul modelelor Mesh [6]. În cadrul sistemului implementat au fost utilizate mai multe tipuri de modele, în funcție de rolul nodului:

- `Generic OnOff Server` — pentru controlul iluminatului
- `Generic OnOff Client` — pentru nodul întrerupător
- `Generic Level Server` — pentru controlul intensității prin `PWM`
- `Sensor Model` — pentru nodul senzor
- `Vendor Model` — pentru transmiterea comenzilor IR

Inițializarea stivei Bluetooth Mesh se realizează la pornirea fiecărui nod [5]. În această etapă sunt configurate elementele esențiale ale rețelei conform specificației [3]:

- `UUID`-ul dispozitivului
- cheile de rețea și aplicație
- modelele asociate nodului [6]
- rolurile (server, client, relay etc.)

După finalizarea procesului de provizionare [3], nodul devine parte a rețelei și poate transmite și recepționa mesaje, conform modelelor configurate. Comunicarea se realizează pe baza adreselor unicast sau de grup, iar mecanismul de retransmitere permite propagarea mesajelor chiar și în absența unei conexiuni directe între noduri [4].

3.6.3. Structura firmware-ului nodurilor

Firmware-ul nodurilor este organizat modular, fiind împărțit într-o componentă comună și componente specifice fiecărui tip de nod [5]. Această abordare permite reutilizarea codului și simplifică dezvoltarea și extinderea sistemului.

Tip nod	**Model Bluetooth Mesh**	**Comenzi / Moduri de lucru**
Întrerupător	`Generic OnOff Client`	`GET`, `SET` (toggle)
Releu iluminat	`Generic OnOff Server`	`ON`, `OFF`
Releu iluminat	`Generic Level Server`	`LEVEL` (0–8192)
Nod IR	`Vendor Model`	`LEARN` (slot 0–4), `SEND` (slot 0–4)
Senzor	`Sensor Server`	`GET` (temperatură, umiditate)
Gateway	`OnOff/Level/Sensor/Vendor Client`	Toate comenzile de mai sus

Tabelul 3.1. Comenzile și modurile de lucru ale nodurilor din rețea

Partea comună include:

- inițializarea stivei Bluetooth Mesh
- gestionarea evenimentelor
- funcții de *publish* și *subscribe*
- mecanisme de reset și configurare

Partea specifică diferă în funcție de rolul nodului și include:

- citirea butoanelor (nod întrerupător)
- controlul `PWM` (nod releu pentru iluminat)
- transmiterea codurilor IR (nod IR)
- citirea valorilor de la senzori (nod senzor)
- procesarea comenzilor primite din rețea

Execuția firmware-ului este bazată pe un model orientat pe evenimente, utilizând task-urile `FreeRTOS` [12] disponibile în `ESP-IDF`. După inițializarea hardware și a stivei Mesh [3], nodul intră într-o buclă principală în care așteaptă apariția evenimentelor.

Fluxul general de execuție al unui nod este:

1. **Inițializare Sistem (Hardware & Software):** Configurarea perifericelor specifice (`GPIO` pentru butoane, `LEDC` pentru `PWM` [2], `RMT` pentru IR [7]). Inițializarea memoriei nevolatile (`NVS`) pentru restaurarea stărilor anterioare și a cheilor de rețea.
2. **Lansare Task-uri de Gestiune (Background Tasks) [12]:** Crearea task-ului de monitorizare a butonului de reset (pentru factory reset). Lansarea task-urilor specifice (ex: task-ul de citire senzori la intervale fixe sau task-ul `RX` pentru captura `IR RAW` [7]).
3. **Activare Stivă Bluetooth Mesh [3]:** Configurarea `UUID`-ului unic și a compoziției elementelor. Înregistrarea funcțiilor de tip *callback* pentru tratarea asincronă a pachetelor primite din rețea.
4. **Așteptare Evenimente (Asincron) [12]:** Nodul intră într-o stare de așteptare eficientă, fiind „trezit” doar de întreruperi hardware sau de pachete radio recepționate.
5. **Procesare și Execuție:**
   - *Eveniment Mesh* [3]: Decodificarea pachetului și acționarea actuatorului (ON/OFF, `PWM` sau IR [7]).
   - *Eveniment Hardware*: Detectarea unei apăsări de buton sau atingerea unui prag de temperatură [15] și publicarea stării în rețeaua Mesh.

Această structură permite reacția rapidă la evenimente și menține un consum redus de resurse [12], deoarece nodurile nu execută operații inutile în absența unor evenimente.

Algoritmul 3.1 Pseudo-cod pentru fluxul logic și tratarea evenimentelor în firmware

```
1:  procedure INITIALIZARE_SISTEM
2:      Init_Hardware(); // Periferice: GPIO, PWM, RMT, I2C
3:      Init_NVS(); // Restaurare date din memoria Flash
4:      Init_Bluetooth_Mesh(); // Inregistrare callback-uri si pornire stiva
5:      Creaza_Task(Monitorizare_Buton_Reset);
6:      if este_nod_senzor then
7:          Creaza_Task(Citire_Periodica_Senzor);
8:  Handler asincron pentru mesaje din retea (Mesh Callback):
9:  procedure LA_RECEPTIE_MESAJ_MESH(mesaj)
10:     Decodifica_Opcode(mesaj);
11:     if Opcode == GENERIC_ONOFF_SET then
12:         Actioneaza_Releu(mesaj.valoare);
13:         Actualizeaza_Stare_Interna();
14:     else if Opcode == GENERIC_LEVEL_SET then
15:         Ajusteaza_PWM_Lumina(mesaj.nivel);
16:     else if Opcode == VENDOR_IR_SEND then
17:         Transmite_Semnal_IR_RAW(mesaj.slot);
18:     Publica_Status_Actualizat(); // Confirmare catre Gateway
19: Handler pentru intreruperi hardware (Butoane/Senzori):
20: procedure LA_APASARE_BUTON_FIZIC
21:     if este_nod_intrerupator then
22:         Cere_Status_Bec_Target(); // Logica Get-before-Set
23:         Asteapta_Raspuns();
24:         Trimite_Comanda_Toggle();
```

3.6.4. Software-ul gateway-ului

Gateway-ul reprezintă legătura dintre rețeaua Bluetooth Mesh și aplicația software de control [3]. Pe gateway rulează un program realizat folosind `ESP-IDF` [5], care primește mesaje prin `MQTT` [9] și le transmite în rețeaua Mesh, respectiv recepționează mesaje din rețea și le trimite către aplicație.

Gateway-ul este conectat la rețeaua locală prin Wi-Fi și rulează un client `MQTT` [9] care se conectează la brokerul `Mosquitto` [14] instalat pe placa `Raspberry Pi` [16].

Rolurile gateway-ului sunt:

- *subscribe* la topicul `mesh/in` [9]
- *publish* pe topicul `mesh/out` [9]
- conversie `MQTT` → Mesh [3]
- conversie Mesh → `MQTT` [9]
- interpretarea mesajelor în format `JSON` [17]

Mesajele primite de la aplicație conțin informații despre comanda care trebuie executată și adresa nodului destinație. Gateway-ul decodează mesajul folosind biblioteca `cJSON` [17] și generează comanda corespunzătoare pentru modelul Bluetooth Mesh utilizat [6].

În sens invers, mesajele primite din rețeaua Mesh sunt convertite în format `JSON` și trimise către aplicație, unde sunt afișate în interfața grafică sau utilizate de sistemul de automatizare.

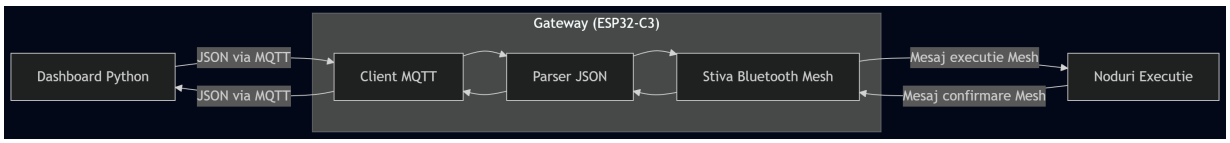


Figura 3.15. Schema logică a Gateway-ului

3.6.5. Protocolul MQTT

Pentru realizarea comunicării între aplicația software și gateway s-a utilizat protocolul `MQTT` [9], datorită simplității și suportului bun pentru sisteme embedded. `MQTT` este un protocol de tip *publish-subscribe*, proiectat pentru aplicații IoT cu resurse limitate [9].

Brokerul `MQTT Mosquitto` [14] rulează pe placa `Raspberry Pi Zero 2 W` [16] și permite schimbul de mesaje între interfața web și gateway-ul `ESP32-C3` [2] conectat la rețeaua Wi-Fi.

Au fost definite două topicuri principale [9]:

- `mesh/in` — utilizat pentru trimiterea comenzilor din aplicație către gateway
- `mesh/out` — utilizat pentru transmiterea mesajelor din rețeaua Mesh către aplicație

Mesajele sunt codificate în format `JSON`, pentru a permite transmiterea mai multor parametri într-un mod flexibil și extensibil. Fiecare comandă conține adresa nodului și tipul acțiunii care trebuie executată.

Această metodă permite extinderea protocolului fără modificarea firmware-ului existent și facilitează integrarea cu alte aplicații software.

3.6.6. Aplicația Flask

Pentru controlul sistemului a fost realizată o aplicație web folosind limbajul Python și framework-ul `Flask` [13]. Aplicația rulează pe placa `Raspberry Pi Zero 2 W` [16] și oferă o interfață grafică accesibilă din browser.

Aplicația permite:

- controlul nodurilor din rețeaua Mesh [3]
- configurarea adreselor de publicare [6]
- vizualizarea datelor primite de la senzori [15]
- definirea scenariilor de automatizare
- monitorizarea mesajelor `MQTT` [9]

Comunicarea cu gateway-ul se realizează prin intermediul unui broker `MQTT Mosquitto` [14], folosind două topicuri [9]: `mesh/in` (pentru trimiterea comenzilor) și `mesh/out` (pentru recepția mesajelor).

În versiunea finală a proiectului, mesajele sunt codificate în format `JSON`, pentru a permite transmiterea mai multor parametri într-un mod standardizat.

Exemplu comandă trimisă către gateway:

```
1  {
2    "cmd": "ON",
3    "addr": "0x0005",
4    "val": 1
5  }
```

Listing 3.4. Exemplu de mesaj JSON pentru comanda de acționare a unui nod

Mesajele primite de la gateway conțin câmpul `type`, care indică tipul informației: `type` = `data`, `event`, `ack`, `error`, `discovery`, `telemetry`.

Aplicația conține un parser JSON care interpretează mesajele primite și actualizează interfața grafică în timp real.

Pentru gestionarea automatizărilor, aplicația include un modul separat care rulează într-un thread independent și verifică periodic condițiile definite de utilizator.

3.6.7. Implementarea scenariilor de automatizare

Aplicația software permite definirea unor scenarii de automatizare care sunt executate fără modificarea firmware-ului nodurilor [3]. Automatizările sunt salvate într-un fișier JSON și sunt interpretate de aplicația Flask [13].

Motorul de automatizare rulează într-un thread separat și verifică periodic condițiile definite. Dacă o condiție este îndeplinită, aplicația trimite automat o comandă către gateway prin MQTT [9].

În implementarea finală au fost definite două tipuri de automatizări:

1. **Automatizări de tip trigger:** Acestea sunt executate atunci când este recepționat un eveniment de la un nod. Exemplu: Dacă temperatura depășește 25°C, se trimite comanda ON către un nod.
2. **Automatizări de tip timer:** Acestea sunt executate periodic, la un interval de timp definit. Exemplu: La fiecare 60 secunde, se trimite comanda OFF către un nod.

Automatizările sunt salvate într-un fișier de configurare și sunt încărcate la pornirea aplicației Flask [13], fără a fi necesară recompilarea firmware-ului [5].

Avantajele acestei metode sunt:

– flexibilitate ridicată
– posibilitatea definirii regulilor din interfața web
– execuție automată fără intervenția utilizatorului
– posibilitatea de automatizări independente de noduri
– extindere ușoară a sistemului

Prin această abordare, logica de automatizare este mutată în aplicația software, iar nodurile Mesh rămân simple, având doar rolul de a executa comenzile primite.

3.6.8. Avantaje și dezavantaje ale tehnologiilor alese

Tehnologiile software utilizate în cadrul proiectului au fost alese astfel încât să permită realizarea unui sistem domotic flexibil, fără dependență de servicii externe și cu posibilitatea extinderii ulterioare. Utilizarea Bluetooth Mesh [3] împreună cu ESP-IDF [5], MQTT [9] și Flask [13] oferă un nivel ridicat de control asupra funcționării sistemului, însă implică și un grad mai mare de complexitate în implementare.

Principalele avantaje ale soluției propuse sunt:

– control complet asupra sistemului, fără utilizarea unor platforme cloud externe [1]
– funcționare locală, independentă de conexiunea la internet
– arhitectură modulară, care permite adăugarea de noi noduri fără modificări majore [3]
– compatibilitate cu standardul Bluetooth Mesh [3], ceea ce asigură interoperabilitate și scalabilitate [4]
– cost redus al componentelor hardware utilizate [2]
– posibilitatea integrării cu alte protocoale prin gateway (MQTT [9], HTTP)

Pe de altă parte, utilizarea acestor tehnologii implică și anumite dezavantaje:

– complexitate ridicată a implementării, în special pentru configurarea rețelei Mesh [3]
– necesitatea procesului de provizionare pentru fiecare nod nou [3]

– dificultate mai mare în depanare, din cauza comunicației descentralizate între noduri [10]
– volum mai mare de cod comparativ cu soluțiile bazate pe platforme comerciale
– necesitatea configurării manuale a adreselor, cheilor și modelelor Mesh [6]

În concluzie, tehnologiile software alese permit realizarea unui sistem domotic complet, stabil și extensibil, potrivit pentru aplicații locale și pentru proiecte experimentale, oferind un control ridicat asupra funcționării, în schimbul unei complexități mai mari în dezvoltare.

3.7. Implementarea soluției

În această secțiune este prezentată implementarea efectivă a sistemului, prin descrierea fiecărui tip de nod realizat. Toate nodurile sunt bazate pe microcontrolerul `ESP32-C3` [2] și folosesc aceeași structură generală de firmware, diferența fiind dată de perifericele conectate și de modelele Bluetooth Mesh utilizate [6].

Pentru fiecare nod au fost realizate: schema electrică, firmware specific, configurare Mesh, testare individuală și integrarea în rețea.

3.7.1. Nod întrerupător

Nodul întrerupător are rolul de a trimite comenzi către alte noduri din rețea [3]. Acesta nu controlează direct un dispozitiv, ci publică mesaje în rețeaua mesh folosind modelul `Generic OnOff Client` [6].

Sistemul a fost montat pe un breadboard, utilizând 2 butoane, unul pentru `reset` și unul pentru controlul propriu-zis.

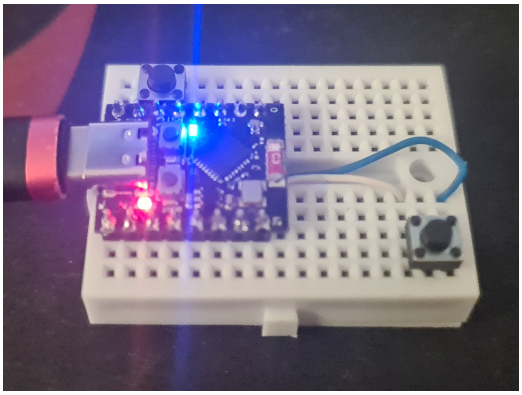


Figura 3.16. Nod Întrerupător fizic

Butoanele sunt conectate la pini `GPIO` configurați ca intrare cu *pull-up* intern [2]. La apăsare, pinul trece în nivel logic `LOW` și firmware-ul detectează evenimentul. Această implementare realizează debouncing prin eșantionare la interval fix de 20 ms. Deși suficientă pentru scenariile testate, o implementare mai robustă ar presupune confirmarea stării stabile pe mai multe eșantionări consecutive, eliminând complet perturbațiile mecanice ale butonului [20]. Codul pentru citirea butonului:

```
1  void button_tap_task(void *arg) {
2      gpio_config_t io_conf = {
3          .intr_type = GPIO_INTR_DISABLE,
4          .mode = GPIO_MODE_INPUT,
5          .pin_bit_mask = (1ULL << BUTTON_IO),
6          .pull_up_en = 1
7      };
8      gpio_config(&io_conf);
9      uint8_t last_st = 1;
```

```
10
11      while (1) {
12          uint8_t curr_st = gpio_get_level(BUTTON_IO);
13          if (last_st == 1 && curr_st == 0) {
14              ESP_LOGI(TAG, "Buton apasat (GPIO 9)");
15              example_ble_mesh_send_gen_onoff_set();
16              vTaskDelay(pdMS_TO_TICKS(500));
17          }
18          last_st = curr_st;
19          vTaskDelay(pdMS_TO_TICKS(20));
20      }
21  }
```

Listing 3.5. Implementarea task-ului FreeRTOS pentru monitorizarea și debouncing-ul butonului fizic

Configurarea adresei de publicare se realizează din interfața grafică [3], permițând modificarea nodului controlat de către întrerupător foarte ușor.

Spre deosebire de un întrerupător clasic, acest nod implementează o logică de sincronizare activă. La detectarea unei apăsări, nodul nu trimite direct o comandă de „*Switch*”, ci parcurge următorii pași:

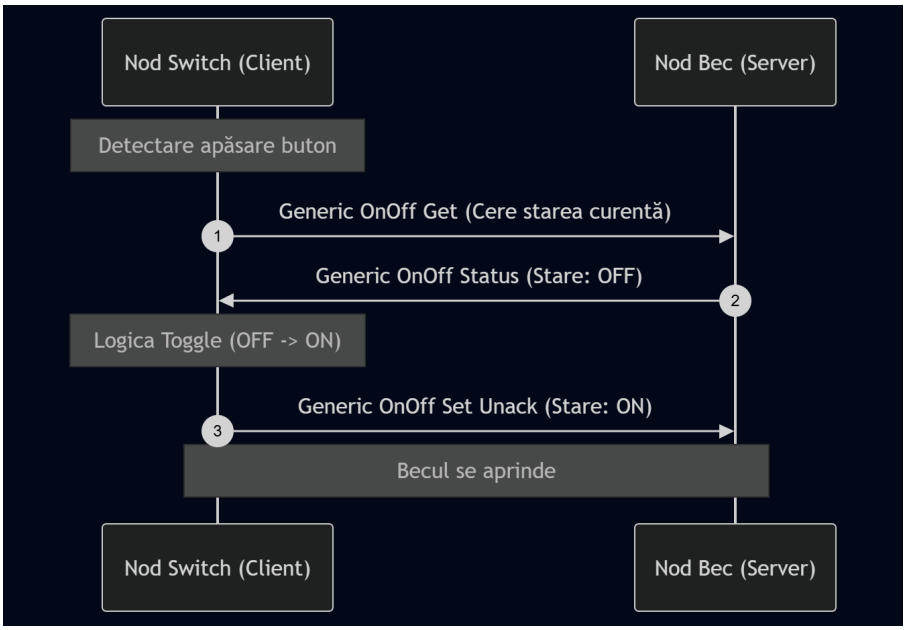


Figura 3.17. Interacțiune Nod Client – Nod Server

Nodul este configurat ca: Generic OnOff Client, Relay activ și TTL = 3.

Acest lucru permite ca mesajul să fie retransmis de alte noduri [10] și să ajungă la destinație chiar dacă nu există legătură directă.

3.7.2. Nod pentru controlul iluminatului

Nodul este utilizat pentru controlul iluminatului sau al altor consumatori de putere. Acesta primește mesaje din rețeaua Bluetooth Mesh [3] și controlează un `MOSFET` [18] conectat la ieșire, permițând comutarea sarcinii.

Sistemul a fost montat pe un breadboard, utilizând un buton pentru `reset` și un modul dual `MOSFET`.

Utilizarea `MOSFET`-ului [18] permite comutarea unor curenți mai mari decât cei suportați direct

de pinii microcontrolerului, fiind astfel posibil controlul unor consumatori precum benzi LED sau LED-uri de putere.

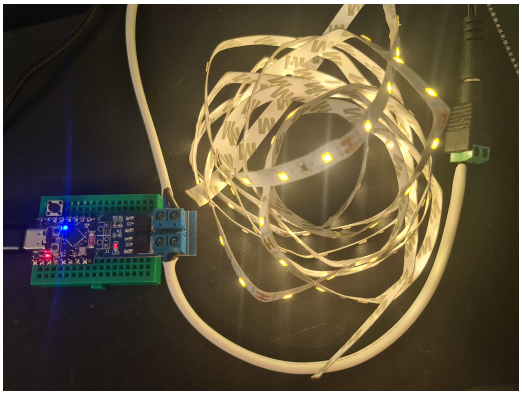


Figura 3.18. Nod control iluminat fizic

Pentru controlul intensității luminii a fost utilizat un semnal PWM generat de ESP32-C3 [2]. Acest semnal comandă MOSFET-ul și permite reglarea progresivă a luminozității. În implementare s-a ales o rezoluție de 13 biți, ceea ce permite până la 8192 de niveluri distincte de intensitate.

Pentru implementarea funcționalității Bluetooth Mesh s-au utilizat modelele standard oferite de Espressif, respectiv `Generic OnOff Server` și `Generic Level Server` [6], integrate într-un proiect unitar. Astfel, nodul poate răspunde atât la comenzi de tip ON/OFF, cât și la comenzi de reglare a nivelului.

Inițializarea modulului PWM se realizează la pornirea sistemului:

```
1   void led_pwm_init() {
2       // 1. Configurare Timer LEDC
3       ledc_timer_config_t ledc_timer = {
4           .speed_mode       = LEDC_MODE,
5           .timer_num        = LEDC_TIMER,
6           .duty_resolution  = LEDC_DUTY_RES,
7           .freq_hz          = LEDC_FREQUENCY,
8           .clk_cfg          = LEDC_AUTO_CLK,
9       };
10      ESP_ERROR_CHECK(ledc_timer_config(&ledc_timer));
11
12      // 2. Configurare Canal LEDC
13      ledc_channel_config_t ledc_channel = {
14          .speed_mode     = LEDC_MODE,
15          .channel        = LEDC_CHANNEL,
16          .gpio_num       = LEDC_OUTPUT_IO,
17          .intr_type      = LEDC_INTR_DISABLE,
18          .timer_sel      = LEDC_TIMER,
19          .duty           = 0,
20          .hpoint          = 0
21      };
22      ESP_ERROR_CHECK(ledc_channel_config(&ledc_channel));
23      ESP_LOGI("PWM", "LEDC PWM initialized on GPIO %d (Max Duty: %u)",
        ↪  LEDC_OUTPUT_IO, LEDC_MAX_DUTY);
24  }
```

Listing 3.6. Inițializarea perifericului LEDC pentru generarea semnalului PWM

Setarea intensității luminoase se face prin actualizarea factorului de umplere al semnalului PWM:

```
static uint32_t current_pwm_duty = 0;

void set_pwm_duty(uint32_t duty) {
    if (duty > LEDC_MAX_DUTY) {
        duty = LEDC_MAX_DUTY;
    }
    current_pwm_duty = duty;
    ledc_set_duty(LEDC_MODE, LEDC_CHANNEL, current_pwm_duty);
    ledc_update_duty(LEDC_MODE, LEDC_CHANNEL);
    ESP_LOGI("PWM", "Duty Cycle set to %u", current_pwm_duty);
}
```

Listing 3.7. Funcție pentru actualizarea factorului de umplere și aplicarea noului nivel PWM

La primirea unui mesaj din rețea, acesta este procesat în funcția de callback asociată modelului Mesh:

```
static void
 ↪ example_ble_mesh_generic_server_cb(esp_ble_mesh_generic_server_cb_event_t
 ↪ event, esp_ble_mesh_generic_server_cb_param_t *param)
{
    switch (event) {
    case ESP_BLE_MESH_GENERIC_SERVER_RECV_GET_MSG_EVT:
        // Solicitare stare curenta a nodului
        if (param->ctx.recv_op == ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_GET)
         ↪ {
            example_handle_gen_onoff_msg(param->model, &param->ctx,
             ↪ NULL);
        }
        else if (param->ctx.recv_op ==
         ↪ ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_GET) {
            example_handle_gen_level_msg(param->model, &param->ctx,
             ↪ NULL);
        }
        break;

    case ESP_BLE_MESH_GENERIC_SERVER_RECV_SET_MSG_EVT:
        // Receptionare comanda de control (OnOff sau Level)
        if (param->ctx.recv_op == ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_SET
            param->ctx.recv_op ==
             ↪ ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_SET_UNACK) {
            // Executa comanda ON/OFF pe MOSFET
            example_handle_gen_onoff_msg(param->model, &param->ctx,
             ↪ &param->value.set.onoff);
        }
        else if (param->ctx.recv_op ==
         ↪ ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_SET
                 param->ctx.recv_op ==
                  ↪ ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_SET_UNACK) {
            // Ajusteaza factorul de umplere PWM (Dimming)
            example_handle_gen_level_msg(param->model, &param->ctx,
             ↪ &param->value.set.level);
```

```
25              }
26              break;
27          default:
28              break;
29          }
30      }
```

Listing 3.8. Funcția de callback pentru tratarea evenimentelor de tip Generic Server (Get și Set)

În funcție de tipul mesajului primit:

- mesaj `ON/OFF` → se comută starea `MOSFET`-ului
- mesaj `LEVEL` → se modifică semnalul `PWM` și implicit intensitatea luminii

În testele realizate, nodul a demonstrat funcționare stabilă, fiind capabil să controleze fără probleme benzi `LED` și `LED`-uri de putere.

Un avantaj important al acestei implementări este flexibilitatea controlului: nodul poate fi comandat atât de la noduri de tip întrerupător, cât și din aplicația software prin intermediul gateway-ului, fără a necesita modificări ale firmware-ului.

3.7.3. Nod IR (telecomandă universală)

Nodul IR are rolul de a controla dispozitive care utilizează telecomandă cu infraroșu. Acesta permite atât învățarea semnalelor de la telecomenzi existente, cât și retransmiterea acestora la primirea unor comenzi din rețeaua Bluetooth Mesh [3].

Sistemul a fost realizat pe breadboard, utilizând un receptor IR `VS1838B` [19], un `LED` infraroșu comandat prin tranzistor MOSFET `2N7000` cu rezistență de 82Ω în serie, alimentat la 5V și trei butoane fizice. Un buton este utilizat pentru resetarea dispozitivului, unul pentru inițierea procesului de învățare a unui semnal IR, iar cel de-al treilea pentru transmiterea semnalului memorat din ultimul slot utilizat.

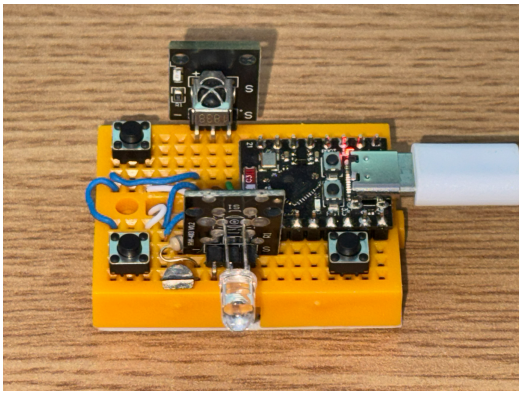


Figura 3.19. Nod IR fizic

Capturarea semnalului IR

Recepția semnalului se realizează asincron prin perifericul `RMT` configurat în modul recepție [7]. La declanșarea procesului de învățare, nodul pornește recepția și așteaptă maxim 5 secunde un semnal valid de la telecomandă. Perifericul capturează autonom secvența de impulsuri și notifică firmware-ul prin callback-ul `on_receive_done`, care se execută în context de întrerupere (`IRAM_ATTR`) și folosește `vTaskNotifyGiveFromISR` pentru a trezi task-ul de recepție fără a bloca execuția:

```
1   static bool IRAM_ATTR on_receive_done(rmt_channel_handle_t channel,
2                                         const rmt_rx_done_event_data_t
                                           ↪  *edata, void *user_ctx) {
```

```
3       BaseType_t high_task_wakeup = pdFALSE;
4       size_t copy_len = edata->num_symbols;
5
6       memcpy(learned_signal_buf, edata->received_symbols, copy_len *
        ↪  sizeof(rmt_symbol_word_t));
7       learned_signal_len = copy_len;
8
9       vTaskNotifyGiveFromISR(ir_rx_task_handle, &high_task_wakeup);
10      return high_task_wakeup == pdTRUE;
11  }
```

Listing 3.9. Funcția de callback executată în context de întrerupere (ISR) la finalizarea recepției unui semnal IR

Semnalul capturat este reprezentat ca un array de simboluri de tip `rmt_symbol_word_t`, fiecare simbol codificând durata și nivelul logic al unui impuls. Bufferul are o capacitate de 64 de simboluri, suficient pentru majoritatea telecomenzilor comerciale. Semnalul este stocat brut, fără decodare sau interpretare a protocolului — această alegere este deliberată, deoarece permite reproducerea fidelă a oricărui semnal, inclusiv a celor cu protocoale proprietare sau necunoscute.

Inițializarea canalului `RMT` pentru recepție:

```
1   rmt_rx_channel_config_t rx_cfg = {
2       .clk_src = RMT_CLK_SRC_DEFAULT,
3       .gpio_num = RMT_RX_GPIO_NUM,
4       .mem_block_symbols = RMT_BUFFER_SIZE,
5       .resolution_hz = IR_RESOLUTION_HZ,
6       .flags.invert_in = true, // Inversare semnal pentru receptoare
        ↪  active-low
7   };
8   ESP_ERROR_CHECK(rmt_new_rx_channel(&rx_cfg, &rx_channel));
9
10  // Inregistrare callback pentru finalizarea receptiei semnalului
11  rmt_rx_event_callbacks_t rx_callbacks = { .on_recv_done =
     ↪  on_receive_done };
12  ESP_ERROR_CHECK(rmt_rx_register_event_callbacks(rx_channel,
     ↪  &rx_callbacks, NULL));
13  rmt_enable(rx_channel);
```

Listing 3.10. Configurarea perifericului RMT și înregistrarea callback-ului pentru canalul de recepție IR

Stocarea și persistența semnalelor

Semnalul capturat este salvat în memoria nevolatilă `NVS`, organizat în 5 sloturi independente (slot 0–4). Fiecare slot stochează datele semnalului și lungimea acestuia folosind chei `NVS` de forma `sig_d_N` și `sig_l_N`. Slotul activ curent este de asemenea persistent, permițând nodului să reia operarea după repornire fără reconfigurare.

Retransmiterea semnalului

La primirea comenzii de retransmitere, nodul încarcă semnalul din slotul specificat din `NVS` și îl transmite prin canalul `RMT` folosind un encoder de tip *copy*. Acesta reproduce fidel simbolurile brute stocate, aplicând automat modularea purtătoarei de 38 kHz configurată pe canal:

```
1   rmt_tx_channel_config_t tx_cfg = {
2       .clk_src = RMT_CLK_SRC_DEFAULT,
3       .gpio_num = RMT_TX_GPIO_NUM,
4       .mem_block_symbols = 64,
```

```
    .resolution_hz = IR_RESOLUTION_HZ, // 1MHz (1us per tick)
    .trans_queue_depth = 4,
};
ESP_ERROR_CHECK(rmt_new_tx_channel(&tx_cfg, &tx_channel));

// Aplicarea frecventei purtatoare de 38kHz (standard telecomenzi)
rmt_carrier_config_t carrier_cfg = {
    .frequency_hz = IR_CARRIER_FREQ_HZ, // 38000 Hz
    .duty_cycle = 0.33
};
ESP_ERROR_CHECK(rmt_apply_carrier(tx_channel, &carrier_cfg));
rmt_enable(tx_channel);
```

Listing 3.11. Configurarea perifericului RMT pentru transmisie și modularea purtătoarei de 38 kHz

Transmisia efectivă a semnalului dintr-un slot:

```
rmt_transmit_config_t tx_config = { .loop_count = 0, .flags.eot_level = 0 };
rmt_transmit(tx_channel, copy_encoder,
             learned_signal_buf,
             learned_signal_len * sizeof(rmt_symbol_word_t),
             &tx_config);
rmt_tx_wait_all_done(tx_channel, 1000);
```

Listing 3.12. Transmisia efectivă a semnalului dintr-un slot

Controlul nodului `IR` din rețeaua Bluetooth Mesh se realizează printr-un model *vendor* personalizat [6], deoarece modelele standard Mesh nu permit transmiterea parametrilor specifici necesari — acțiunea dorită și numărul slotului. Modelul definește un opcode de intrare (`VND_MODEL_OP_SEND`) și un opcode de status (`VND_MODEL_OP_STATUS`) pentru feedback.

Mesajul de comandă conține doi octeți: primul indică acțiunea (1 = învățare, 2 = retransmitere), al doilea indică slotul țintă (0–4). Feedback-ul este implementat în două moduri în funcție de momentul disponibilității răspunsului:

- **Feedback sincron** — confirmare imediată trimisă în același context de callback Mesh, folosind contextul original al mesajului primit (de exemplu, `STATUS_LEARN_STARTED` la inițierea învățării)
- **Feedback asincron** — trimis după finalizarea operației, folosind un context Mesh reconstruit explicit cu adresa fixă a gateway-ului, deoarece contextul original al tranzacției a expirat

```
static void vendor_model_cb(esp_ble_mesh_model_cb_event_t event,
    esp_ble_mesh_model_cb_param_t *param) {
    if (event == ESP_BLE_MESH_MODEL_OPERATION_EVT &&
        param->model_operation.opcode ==
            ESP_BLE_MESH_VND_MODEL_OP_SEND) {

        uint8_t action = param->model_operation.msg[0]; // 1=Learn,
            2=Send
        uint8_t slot   = param->model_operation.msg[1]; // Slotul NVS
            (0-4)

        if (action == 1) { // Logica Invatare
```

```
 9                active_slot = slot;
10                send_mesh_status(STATUS_LEARN_STARTED, slot, false); //
                   ↪ Confirmare rapida
11                xTaskNotifyGive(ir_rx_task_handle); // Trigger task de
                   ↪ receptie
12            } else if (action == 2) { // Logica Redare
13                transmit_ir_from_slot(slot, true);
14            }
15        }
16    }
```

Listing 3.13. Funcția de callback pentru modelul Vendor Mesh și decodarea comenzilor

Codurile de status definite sunt:

Cod	Constantă	Semnificație
0x01	`STATUS_LEARN_STARTED`	Procesul de învățare a început
0x02	`STATUS_LEARN_SUCCESS`	Semnal capturat și salvat cu succes
0x03	`STATUS_LEARN_TIMEOUT`	Timeout — niciun semnal recepționat în 5 s
0x04	`STATUS_SEND_SUCCESS`	Semnal retransmis cu succes
0x05	`STATUS_SEND_ERR_EMPTY`	Slotul specificat este gol

Tabelul 3.2. Coduri status Vendor Model

Fluxul complet de funcționare al nodului `IR` este ilustrat în Figura 3.20.

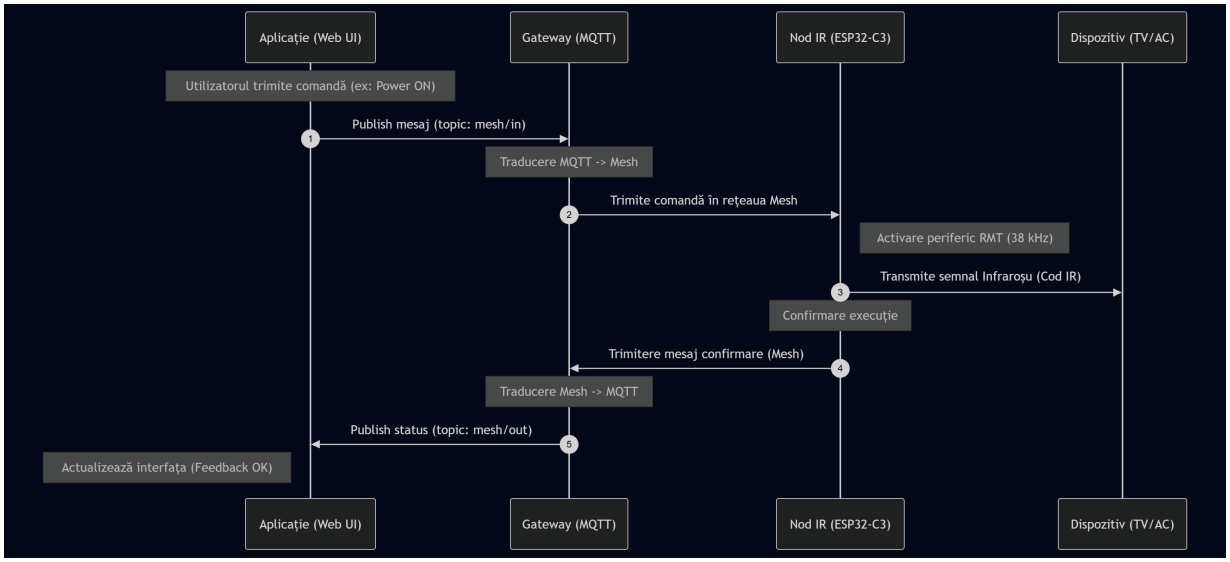


Figura 3.20. Diagramă secvențială IR

Prin utilizarea acestui nod, dispozitivele clasice controlate prin telecomandă pot fi integrate în sistemul smart home fără modificări hardware asupra acestora. Suportul pentru 5 sloturi independente permite controlul mai multor dispozitive sau comenzi diferite de pe același nod, iar persistența în NVS asigură păstrarea semnalelor memorate după repornirea dispozitivului.

3.7.4. Nod senzor temperatură și umiditate

Pentru monitorizarea parametrilor de mediu a fost realizat un nod senzor bazat pe `ESP32-C3` [2] și senzorul de temperatură și umiditate `DHT22` (`AM2302`) [15]. Nodul permite citirea periodică a valorilor și transmiterea acestora în rețeaua Bluetooth Mesh [3] la cererea gateway-ului.

Sistemul a fost realizat pe breadboard, utilizând modulul AM2302 și o rezistență de *pull-up* pe linia de date, necesară pentru funcționarea protocolului digital al senzorului.

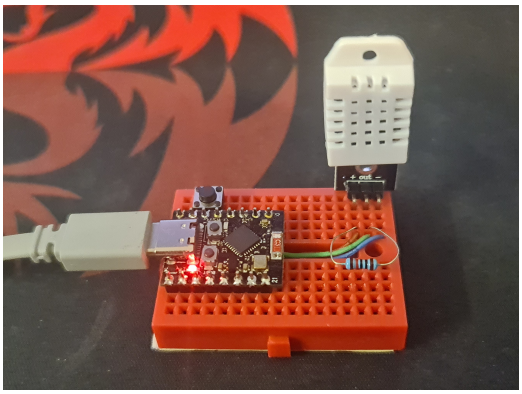


Figura 3.21. Nod senzor fizic

Citirea senzorului se face folosind un driver software dedicat pentru DHT22 [15], inclus în proiect sub forma fișierelor dht.c și dht.h. Driverul utilizează întreruperi GPIO și timerul hardware al microcontrolerului [2] pentru măsurarea duratei impulsurilor generate de senzor, conform protocolului specific acestuia.

Datele sunt citite într-un task FreeRTOS care rulează periodic la 10 secunde.

```
1   // --- TASK CITIRE DHT22 ---
2   void DHT_mesh_task(void *pvParameter) {
3       uint16_t temperature, humidity;
4       DHTinit(PIN_DHT22);
5       vTaskDelay(2000 / portTICK_PERIOD_MS);
6
7       while(1) {
8           int r = DHTget(&temperature, &humidity);
9           if(r >= 0) {
10              // Conversie pentru Mesh Precizie Inalta (scara 1/100)
11              // DHT returneaza 258 pentru 25.8, Mesh trimitem 2580
                ↪  (25.80)
12              current_temp_mesh = (int16_t)(temperature * 10);
13              current_humid_mesh = (uint16_t)(humidity * 10);
14
15              net_buf_simple_reset(&sensor_data_temp);
16              net_buf_simple_add_le16(&sensor_data_temp,
                ↪  current_temp_mesh);
17
18              net_buf_simple_reset(&sensor_data_humid);
19              net_buf_simple_add_le16(&sensor_data_humid,
                ↪  current_humid_mesh);
20
21              // local display
22              ESP_LOGI(TAG, "Senzor Update: T=%d.%dC, H=%d.%d%%",
23                       temperature/10, temperature%10,
24                       humidity/10, humidity%10);
25          }
26          vTaskDelay(10000 / portTICK_PERIOD_MS);
27      }
28  }
```

Listing 3.14. Task FreeRTOS pentru citirea periodică a senzorului DHT22 și scalarea datelor pentru formatul Mesh

Valorile temperaturii și umidității sunt convertite într-un format compatibil cu specificația Bluetooth Mesh Sensor Model [6], folosind reprezentare pe 2 octeți pentru fiecare valoare, pentru a permite transmiterea cu precizie mai mare.

Transmiterea în rețea se face doar la cererea gateway-ului, prin mesajul `Sensor Get` [6], pentru a evita traficul inutil în rețea.

```
1   static void
     ↪  example_ble_mesh_sensor_server_cb(esp_ble_mesh_sensor_server_cb_event_t
     ↪  event, esp_ble_mesh_sensor_server_cb_param_t *param) {
2       if (event == ESP_BLE_MESH_SENSOR_SERVER_RECV_GET_MSG_EVT) {
3           if (param->ctx.recv_op == ESP_BLE_MESH_MODEL_OP_SENSOR_GET) {
4               ESP_LOGI(TAG, "Gateway a cerut datele senzorului...");
5
6               uint8_t status_msg[12];
7               uint16_t len = 0;
8
9               // Temperatura (Format B, 2 octeti date: (1 << 1) | 1)
10              status_msg[len++] = (1 << 1) | 1;
11              status_msg[len++] = (uint8_t)(SENSOR_PROPERTY_ID_TEMP &
                 ↪  0xFF);
12              status_msg[len++] = (uint8_t)(SENSOR_PROPERTY_ID_TEMP >>
                 ↪  8);
13              status_msg[len++] = (uint8_t)(current_temp_mesh & 0xFF);
14              status_msg[len++] = (uint8_t)(current_temp_mesh >> 8);
15
16              // Umiditatea (Format B, 2 octeti date)
17              status_msg[len++] = (1 << 1) | 1;
18              status_msg[len++] = (uint8_t)(SENSOR_PROPERTY_ID_HUMID &
                 ↪  0xFF);
19              status_msg[len++] = (uint8_t)(SENSOR_PROPERTY_ID_HUMID >>
                 ↪  8);
20              status_msg[len++] = (uint8_t)(current_humid_mesh & 0xFF);
21              status_msg[len++] = (uint8_t)(current_humid_mesh >> 8);
22
23              esp_ble_mesh_server_model_send_msg(param->model,
                 ↪  &param->ctx, ESP_BLE_MESH_MODEL_OP_SENSOR_STATUS, len,
                 ↪  status_msg);
24          }
25      }
26  }
```

Listing 3.15. Tratarea cererilor de tip Sensor Get și asamblarea pachetului de status cu datele de mediu

Fluxul de funcționare este:

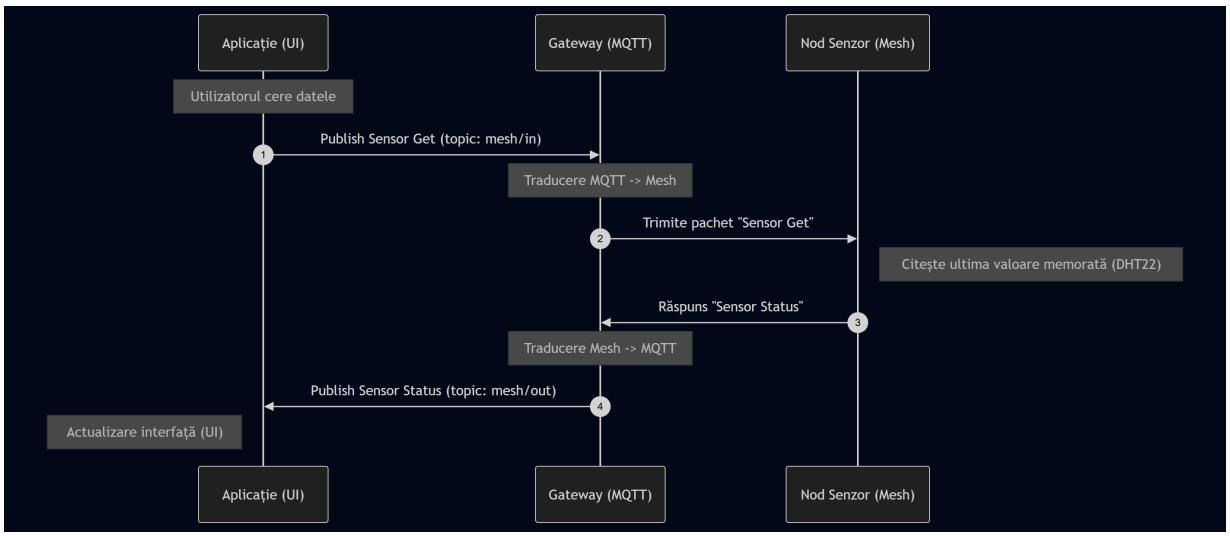


Figura 3.22. Diagramă secvențială senzor

Valorile sunt afișate în interfața aplicației și pot fi utilizate în scenarii de automatizare.

Exemplu: Dacă temperatura este mai mare de 25°C, se trimite comandă către nodul `IR` și se pornește aerul condiționat.

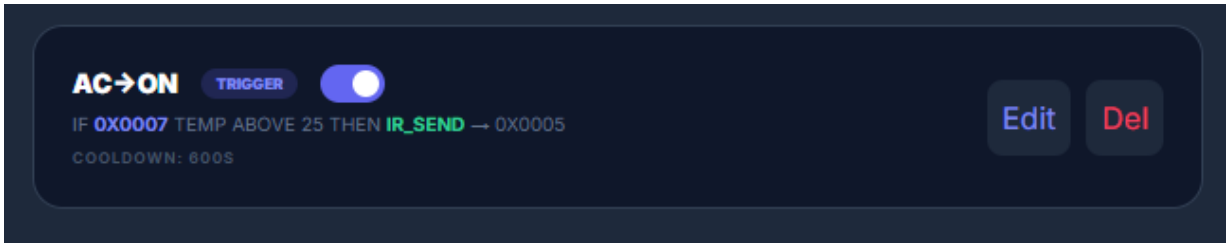


Figura 3.23. Automatizare aer condiționat

3.7.5. Nod gateway și aplicația de control

Gateway-ul este componenta care face legătura între rețeaua Bluetooth Mesh și aplicația software [3]. Acesta este realizat folosind o placă `ESP32-C3 Mini DevKit` [2], deoarece aceasta oferă o stabilitate mai mare decât versiunea SuperMini.

Gateway-ul este configurat ca: `Proxy Mesh` [3], `Provizioner Rețea Mesh` [3] și `Client MQTT` [9].

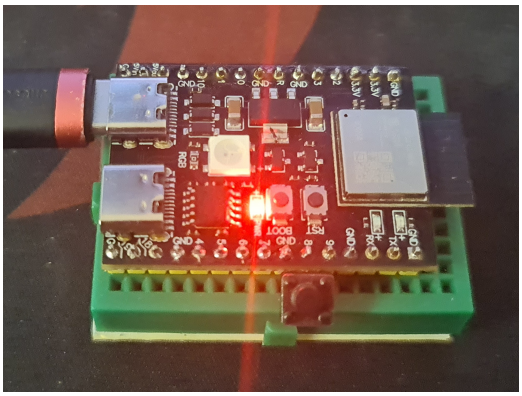


Figura 3.24. Nod Gateway fizic

Procesul de provizionare este inițiat din interfața grafică prin pagina dedicată, prezentată în Figura 4.2. Gateway-ul îndeplinește simultan rolul de `Provizioner` și `Proxy` [3], ceea ce îi permite

atât să admită noduri noi în rețea, cât și să servească drept punct de acces pentru comunicația externă prin GATT. La pornire, stiva Mesh este inițializată doar după stabilirea conexiunii Wi-Fi și obținerea adresei IP — ordinea este impusă de dependența modulului Proxy față de stiva de rețea:

```
// Pornire conditionata de obtinerea adresei IP
static void event_handler(void* arg, esp_event_base_t base,
                          int32_t id, void* event_data) {
    if (base == IP_EVENT && id == IP_EVENT_STA_GOT_IP)
        start_ble_mesh_logic(); // mesh + mqtt pornesc doar dupa Wi-Fi
}
```

Listing 3.16. Tratarea evenimentului de IP obținut pentru pornirea secvențială a stivelor Mesh și MQTT

Modelele locale ale gateway-ului (OnOff Client, Level Client, Sensor Client, Vendor Client) sunt legate de AppKey imediat după adăugarea acesteia în stivă, prin callback-ul PROVISIONER_ADD_LOCAL_APP_KEY_COMP_EVT, permițând gateway-ului să trimită comenzi către orice nod configurat din rețea.

Comunicarea cu aplicația se realizează prin protocolul MQTT [9] folosind brokerul Mosquitto [14], topicurile utilizate fiind: mesh/in și mesh/out.

```
{
  "cmd": "ON",
  "addr": "0x0005",
  "val": 1
}
```

```
{
  "type": "data",
  "addr": "0x0005",
  "key": "STATE",
  "val": 1
}
```

Listing 3.17. Exemple de mesaje JSON: stânga (intrare prin mesh/in), dreapta (ieșire prin mesh/out)

```
// Initializarea clientului MQTT si inregistrarea handler-ului de date
const esp_mqtt_client_config_t mqtt_cfg = {
    .broker.address.uri = MQTT_URL
};
mqtt_client = esp_mqtt_client_init(&mqtt_cfg);
esp_mqtt_client_register_event(mqtt_client, ESP_EVENT_ANY_ID,
 ↪  mqtt_event_handler, NULL);

// Handler pentru receptia mesajelor de la Dashboard (Python)
static void mqtt_event_handler(void *handler_args, esp_event_base_t
 ↪  base, int32_t event_id, void *event_data) {
    esp_mqtt_event_handle_t event = event_data;
    if (event_id == MQTT_EVENT_DATA) {
        char *msg = malloc(event->data_len + 1);
        if (msg) {
            memcpy(msg, event->data, event->data_len);
            msg[event->data_len] = '\0';
            // Trimite mesajul brut catre parserul JSON
            process_incoming_command(msg);
            free(msg);
        }
    }
}
```

Listing 3.18. Inițializarea clientului MQTT

La recepția unui mesaj MQTT [9], gateway-ul interpretează câmpurile JSON și generează comanda corespunzătoare în rețeaua Bluetooth Mesh [3].

```
// Parsarea JSON si generarea comenzilor in reteaua Mesh
void process_incoming_command(const char *raw) {
    cJSON *root = cJSON_Parse(raw);
    if (!root) return;

    cJSON *cmd_item = cJSON_GetObjectItem(root, "cmd");
    const char *cmd = cmd_item->valuestring;

    // Extragere adresa destinatie (hex string "0x0005" -> uint16_t)
    uint16_t addr = (uint16_t)strtol(cJSON_GetObjectItem(root,
     ↪  "addr")->valuestring, NULL, 16);

    if (strcmp(cmd, "ON") == 0) {
        send_mesh_onoff(addr, 1);
    } else if (strcmp(cmd, "OFF") == 0) {
        send_mesh_onoff(addr, 0);
    } else if (strcmp(cmd, "IR_SEND") == 0) {
        uint8_t slot = (uint8_t)cJSON_GetObjectItem(root,
         ↪  "slot")->valuedouble;
        send_ir_command(addr, 2, slot); // Action 2 = SEND
    }
    cJSON_Delete(root);
}
```

Listing 3.19. Procesarea comenzilor JSON

În sens invers, mesajele primite din Mesh sunt convertite în format JSON și publicate pe topicul mesh/out.

```
// Conversia datelor din Mesh in format JSON pentru MQTT
static void publish_json(cJSON *root) {
    char *str = cJSON_PrintUnformatted(root);
    if (str) {
        // Publicare pe topicul de iesire catre Python
        esp_mqtt_client_publish(mqtt_client, TOPIC_OUT, str,
         ↪  strlen(str), 1, 0);
        free(str);
    }
    cJSON_Delete(root);
}

// Exemplu raportare temperatura (Sensor Model -> JSON)
static void json_send_data_f(uint16_t addr, const char *key, float val)
 ↪  {
    cJSON *root = cJSON_CreateObject();
    cJSON_AddStringToObject(root, "type", "data");
    cJSON_AddStringToObject(root, "key", key); // ex: "TEMP"
    cJSON_AddNumberToObject(root, "val", val);
    publish_json(root); // Trimite catre MQTT
}
```

Listing 3.20. Trimiterea răspunsului JSON

Schema completă a mesajelor `JSON` schimbate între gateway și aplicație acoperă mai multe tipuri, fiecare cu un câmp `type` distinct: `data` pentru valori de stare ale nodurilor (`STATE`, `LEVEL`, `TEMP`,

HUMID, IR_STATUS), event pentru evenimente de sistem (CONFIG_DONE, NEW_NODE, NODE_DELETED), unprov pentru dispozitive neprovizionate detectate, discovery pentru capabilitățile unui nod după citirea *Composition Data*, ack pentru confirmarea comenzilor primite și error pentru semnalarea erorilor. Această schemă standardizată permite aplicației Python să proceseze toate mesajele printr-un singur parser JSON și să actualizeze interfața grafică în timp real.

Aplicația de control este realizată în Python folosind framework-ul Flask [13] și rulează pe o placă Raspberry Pi Zero 2 W [16], care funcționează ca server local al sistemului. Pe aceasta sunt instalate sistemul de operare Raspberry Pi OS, brokerul MQTT Mosquitto [14] și mediul de execuție Python. Această arhitectură permite funcționarea complet locală, fără dependență de servicii *cloud* [1].

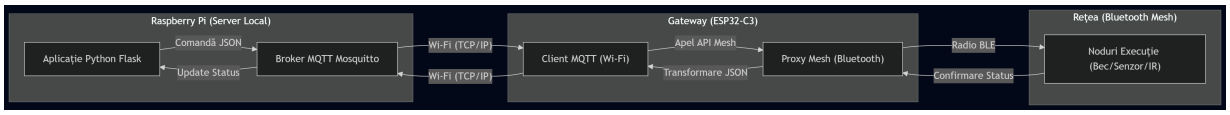


Figura 3.25. Diagramă flux date

Aplicația include următoarele componente: server Flask, client MQTT, interfață web (HTML + JavaScript) și modulul de automatizări.

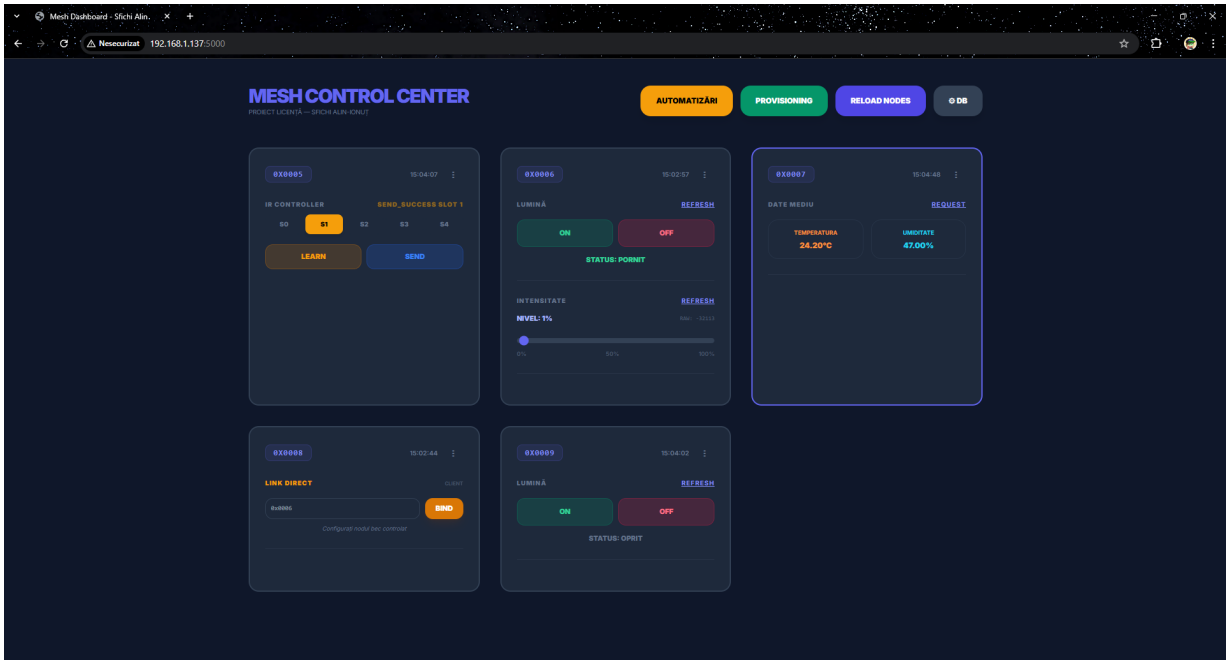


Figura 3.26. Interfața grafică (pagina principală)

Fluxul de comunicare este următorul:

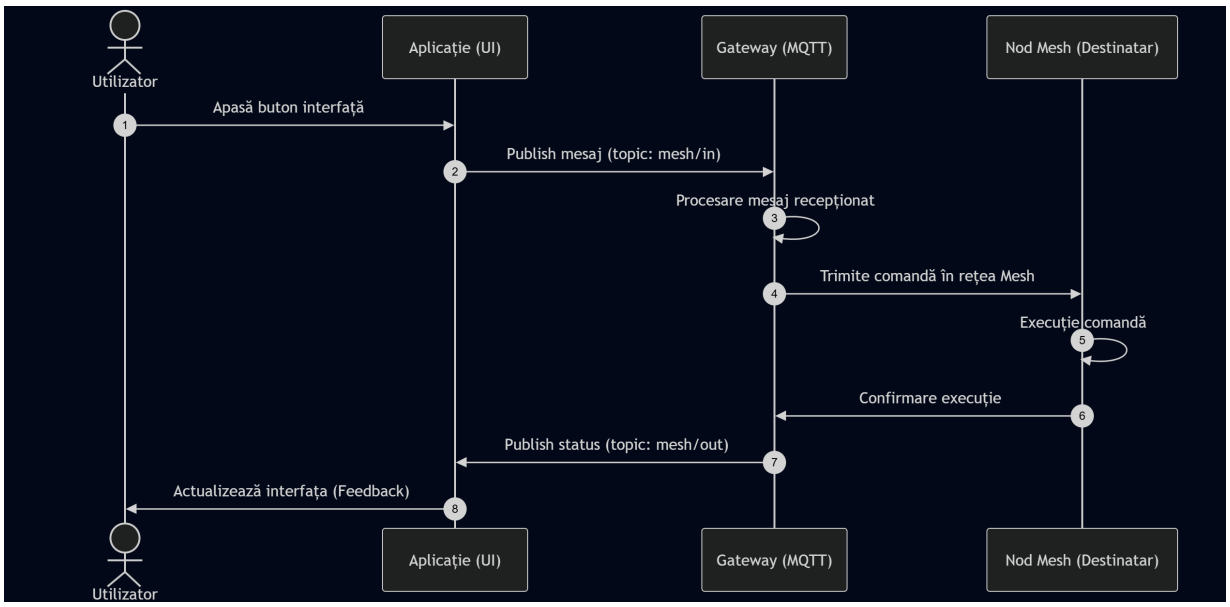


Figura 3.27. Diagramă secvențială comunicare

Aplicația permite definirea de scenarii de automatizare, care pot controla unul sau mai multe noduri simultan. Acestea pot fi declanșate fie în funcție de starea altor dispozitive, fie independent, pe baza unor intervale de timp (*timer*).

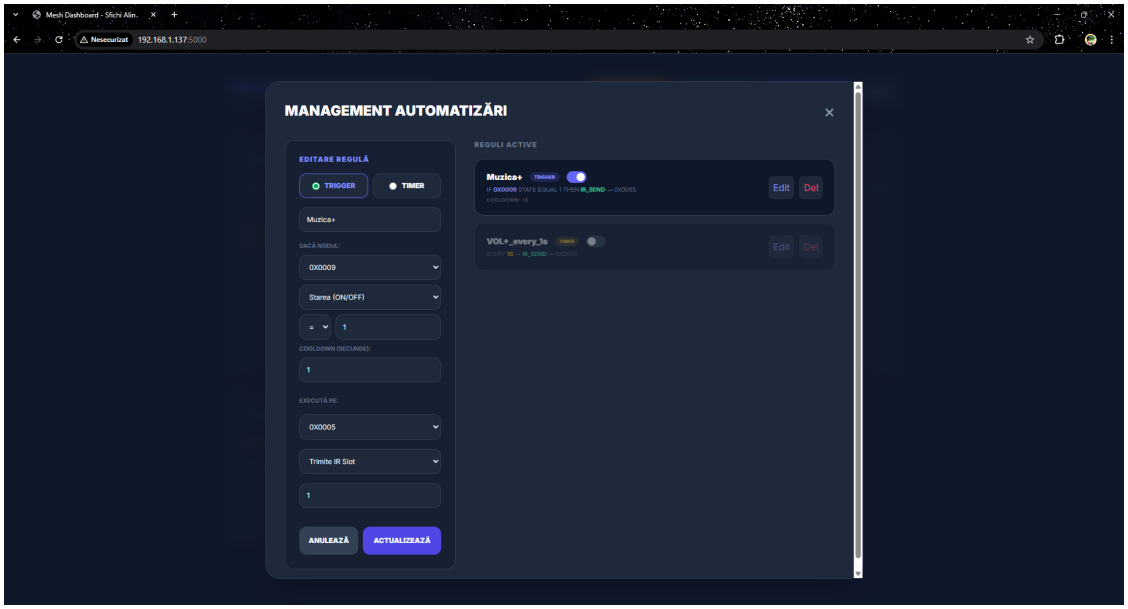


Figura 3.28. Interfața grafică (pagina automatizări)

3.7.6. Probleme întâmpinate și soluții

În timpul dezvoltării sistemului au apărut mai multe probleme legate atât de partea hardware, cât și de partea software, în special în etapa de integrare a tuturor componentelor într-o rețea `Bluetooth Mesh` funcțională [3].

O primă problemă a fost legată de utilizarea plăcii `ESP32-C3 SuperMini` [2]. În timpul testelor s-a observat o instabilitate în funcționarea modulului `Wi-Fi`, în special în cazul nodului gateway, unde conexiunea se realiza greu sau se întrerupea în timpul funcționării.

După analiză s-a constatat că problema este cauzată de proiectarea plăcii, în special de dimensiunea redusă a antenei și de traseele `RF` [2], care pot afecta performanța comunicației wireless.

Pentru rezolvarea acestei probleme, nodul gateway a fost realizat folosind placa ESP32-C3 Mini DevKit [2], care are un design mai stabil și oferă o conexiune Wi-Fi mai fiabilă.

O altă dificultate a fost configurarea rețelei Bluetooth Mesh [3]. Procesul de *provizionare* și configurare este mai complex decât în cazul unei comunicații clasice Bluetooth, deoarece fiecare nod trebuie adăugat în rețea și configurat cu adrese, chei de securitate și modele Mesh [3].

În primele teste, nodurile nu comunicau între ele din cauza configurării incorecte a adreselor de publicare și de abonare. Problema a fost rezolvată prin utilizarea corectă a modelului Configuration Client și verificarea parametrilor setați pentru fiecare nod.

Probleme au apărut și la implementarea comunicației dintre gateway și aplicația software [9]. Gateway-ul transmite datele folosind protocolul MQTT [9], iar aplicația Flask [13] trebuie să se aboneze la topicurile corecte pentru a primi mesajele. În anumite situații mesajele nu erau recepționate din cauza configurării greșite a topicurilor sau a ordinii de pornire a serviciilor. După stabilirea unei structuri clare de topicuri și inițializarea corectă a brokerului Mosquitto [14], comunicația a devenit stabilă.

Aplicația software și brokerul MQTT Mosquitto [14] sunt rulate pe un modul Raspberry Pi Zero 2 W [16], care funcționează ca server local pentru sistem. Această configurație permite controlul centralizat fără utilizarea unui serviciu *cloud*, însă a necesitat configurarea manuală a serviciilor și a conexiunii în rețea.

Dificultăți au fost întâmpinate și la implementarea nodului IR [7], deoarece transmiterea codurilor infraroșu necesită o temporizare foarte precisă. În primele versiuni, semnalele generate nu erau recunoscute de dispozitivele controlate. Problema a fost rezolvată prin utilizarea perifericului RMT hardware al microcontrolerului [7] și a timerelor interne [2], care permit generarea semnalului la frecvența standard de aproximativ 38 kHz.

Capitolul 4. Testarea aplicației și rezultate experimentale

4.1. Punerea în funcțiune a sistemului

După implementarea componentelor hardware și software, sistemul a fost testat într-un mediu de laborator pentru a verifica funcționarea corectă a tuturor modulelor. Punerea în funcțiune a presupus configurarea nodurilor `Bluetooth Mesh` [3], instalarea aplicației software și realizarea conexiunii dintre gateway și interfața de control.

Primul pas a fost instalarea brokerului `MQTT Mosquitto` [14] pe placa `Raspberry Pi Zero 2 W` [16], utilizată ca server local al sistemului. Pe această placă rulează sistemul de operare `Linux`, aplicația `Flask` [13] și brokerul `MQTT` [14], care permite schimbul de mesaje între aplicația de control și nodul gateway.

Brokerul a fost configurat folosind setările implicite, iar aplicația `Flask` a fost configurată să se conecteze la acesta folosind cele două topicuri definite pentru comunicația cu gateway-ul.

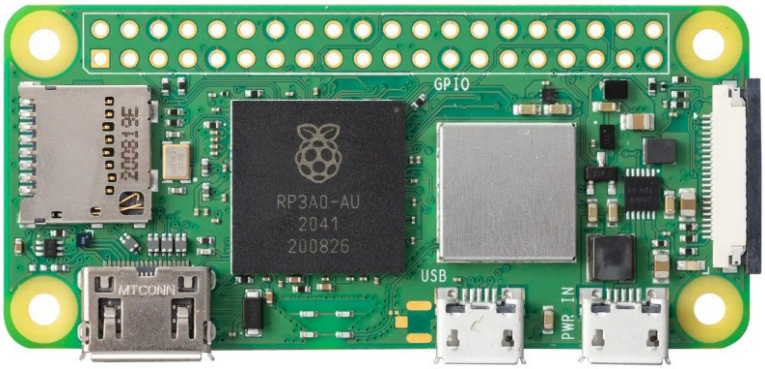


Figura 4.1. Placa de dezvoltare Raspberry Pi Zero 2 W [4]

Al doilea pas a fost programarea firmware-ului pe fiecare placă `ESP32-C3` [2] folosind mediul de dezvoltare `ESP-IDF` [5]. Pentru fiecare tip de nod a fost compilat un firmware diferit, corespunzător rolului acestuia în rețeaua `Bluetooth Mesh` [3].

După încărcarea firmware-ului pe fiecare placă `ESP32-C3` și pornirea sistemului, procesul de *provizionare* se realizează prin interfața grafică, accesibilă din browser la adresa `IP` a plăcii `Raspberry Pi` pe portul 5000. Pagina de *provizionare*, prezentată în Figura 4.2, afișează în timp real lista dispozitivelor neprovizionate detectate de gateway, identificate prin `UUID` și puterea semnalului `RSSI`.

Procesul parcurge următorii pași din perspectiva utilizatorului: dispozitivul neprovizionat este pornit și apare automat în lista de scanare în câteva secunde; utilizatorul apasă butonul „Add” pentru a iniția *provizionarea*; interfața afișează un indicator de progres și confirmă finalizarea cu succes prin afișarea adresei *unicast* atribuite; nodul apare imediat în pagina principală cu capabilitățile sale detectate automat.

Întregul proces de *provizionare* și configurare durează în practică între 3 și 8 secunde per nod, în funcție de numărul de modele prezente și de latența rețelei `Wi-Fi` locale. După finalizare, nodul este persistent în baza de date `NVS` a gateway-ului și nu necesită *re-provizionare* la reporniri ulterioare ale sistemului.

[4] Imagine preluată de la https://www.emag.ro/placa-de-dezvoltare-raspberry-pi-zero-w-v2-ai02084/pd/D5CPY83BM/

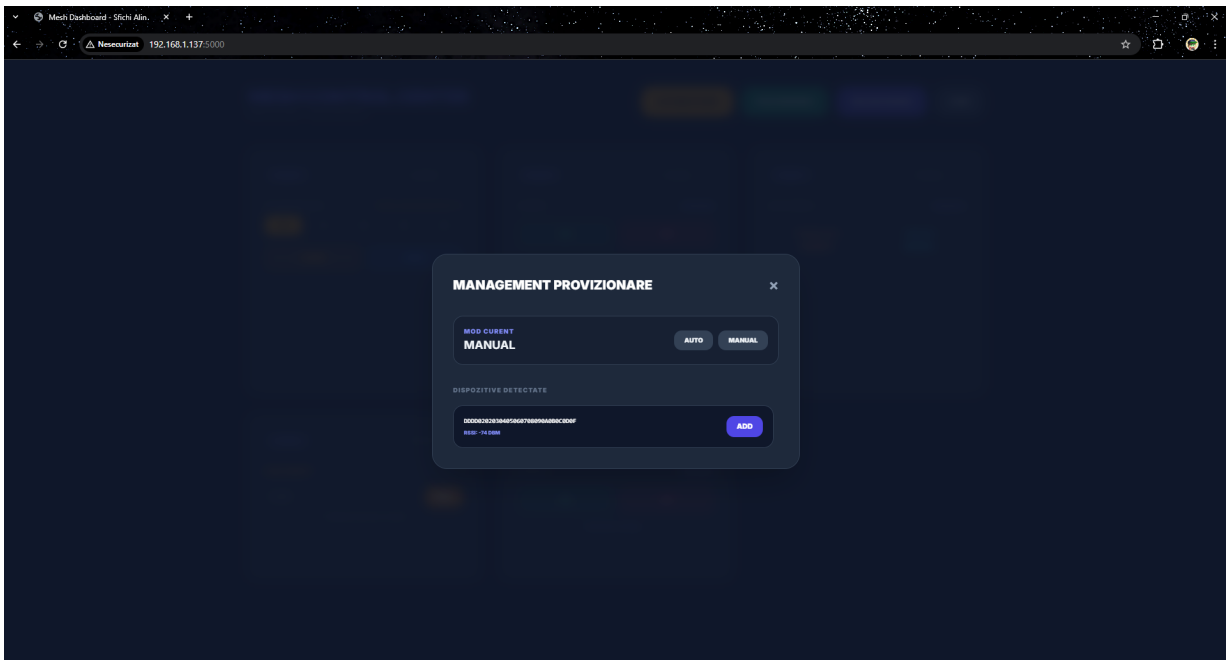


Figura 4.2. Interfața grafică (pagina provizionare)

După finalizarea procesului de provizionare și configurare, nodurile au devenit disponibile în interfața grafică și au putut fi controlate din pagina principală a aplicației.

4.2. Testarea funcțională a nodurilor

Fiecare nod a fost testat individual înainte de integrarea în rețeaua `Bluetooth Mesh`, pentru a verifica funcționarea corectă a componentelor hardware și a firmware-ului [5].

Testarea nodului întrerupător a constat în verificarea detectării apăsării butoanelor și transmiterea mesajelor `Mesh` [3] către adresa configurată. S-a verificat că mesajele sunt generate corect și sunt propagate în rețea, fiind recepționate de nodurile abonate la adresa respectivă.

Testarea nodului pentru controlul iluminatului a constat în trimiterea de comenzi `ON/OFF` și verificarea comutării corecte a sarcinii conectate la ieșirea `MOSFET` [18]. De asemenea, a fost testat controlul `PWM` [2] pentru reglarea intensității luminii, folosind modelul `Generic Level` [6]. S-a observat că modificarea nivelului este aplicată corect și fără întârzieri vizibile.

Testarea nodului `IR` a constat în învățarea codurilor de la o telecomandă și retransmiterea acestora la primirea unei comenzi din rețea. Funcționarea a fost verificată folosind un televizor și un aparat de aer condiționat, confirmând că semnalele sunt generate la frecvența corectă [7] și sunt recunoscute de dispozitive.

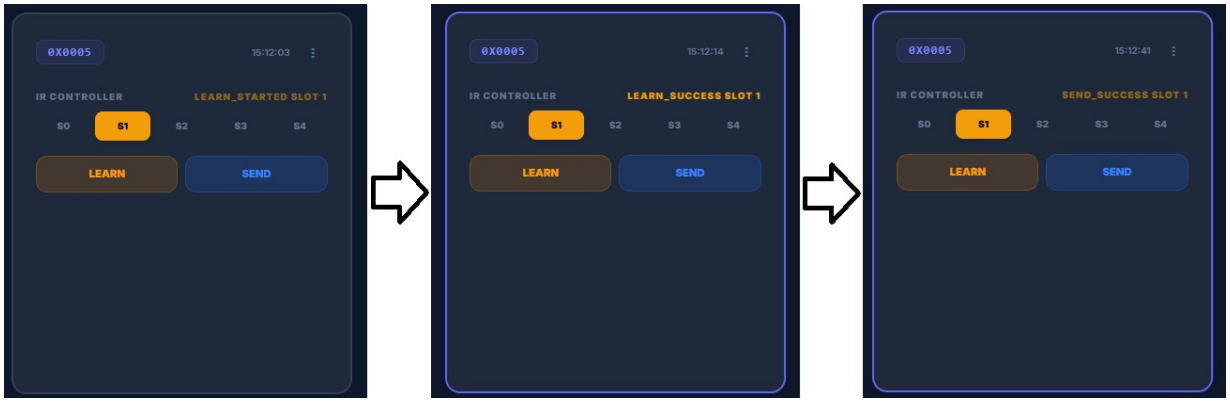


Figura 4.3. Test nod IR

Testarea nodului senzor a constat în citirea valorilor de temperatură și umiditate de la senzorul DHT22 [15] și transmiterea acestora în rețeaua Mesh [3]. Valorile au fost recepționate de gateway, publicate prin MQTT [9] și afișate în aplicația software, confirmând funcționarea corectă a fluxului de date.

Toate nodurile au funcționat corect în testele individuale, fără blocări sau resetări, fiind pregătite pentru integrarea în rețeaua *mesh*.

4.3. Testarea rețelei Bluetooth Mesh

După testarea individuală, nodurile au fost integrate într-o rețea Bluetooth Mesh [3] formată din mai multe dispozitive plasate în încăperi diferite. Pentru nodurile alimentate permanent a fost activat rolul Relay [3], astfel încât acestea să poată retransmite mesajele primite către alte noduri din rețea [10].

S-a testat transmiterea comenzilor între noduri aflate în raza directă de comunicație, precum și între noduri aflate în camere diferite, fără legătură directă. În toate cazurile, mesajele au ajuns la destinație datorită mecanismului de retransmitere (*Managed Flooding*) specific Bluetooth Mesh [4, 10].

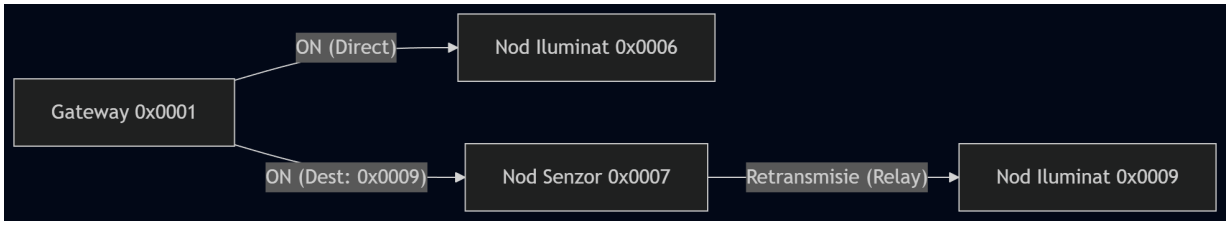


Figura 4.4. Schema logică test retransmisie mesaj

Pentru verificarea funcționării mecanismului de *relay*, s-a dezactivat temporar un nod intermediar și s-a observat că mesajele nu mai ajung la destinație în anumite situații, confirmând rolul important al nodurilor configurate ca Relay.

De asemenea, a fost testată adăugarea de noduri noi în rețea, prin procesul de *provizionare* [3], fără modificarea firmware-ului existent. După configurare, noile noduri au putut comunica cu restul rețelei, confirmând scalabilitatea sistemului.

S-a verificat și comunicarea dintre gateway și nodurile din rețea, comenzile trimise din aplicația software fiind propagate corect prin rețeaua *mesh* către dispozitivele controlate.

Testele realizate au demonstrat că rețeaua Bluetooth Mesh funcționează stabil și permite extinderea sistemului fără modificări majore ale configurației existente [3].

4.4. Testarea comunicării prin gateway și MQTT

Pentru verificarea funcționării controlului centralizat, au fost trimise comenzi din aplicația Flask [13] către gateway, folosind protocolul MQTT [9]. Aplicația Flask și brokerul MQTT Mosquitto [14] rulează pe placa Raspberry Pi Zero 2 W [16], care funcționează ca server local al sistemului.

Gateway-ul realizat cu ESP32-C3 [2] se conectează la brokerul MQTT [14] prin rețeaua Wi-Fi și are rolul de a face legătura între aplicația software și rețeaua Bluetooth Mesh [3]. Mesajele trimise din interfața web sunt publicate pe topicul mesh/in [9], sunt preluate de gateway și retransmise în rețeaua *mesh* sub formă de mesaje Bluetooth Mesh [3].

La rândul lor, nodurile din rețea transmit răspunsuri sau stări către gateway, care le publică pe topicul mesh/out [9], permițând aplicației să actualizeze interfața grafică în timp real.

Gateway-ul a publicat mesajele în rețeaua *mesh*, iar nodurile au executat comenzile corespunzătoare. De asemenea, mesajele generate de noduri au fost trimise înapoi către aplicație prin topicul mesh/out, confirmând funcționarea bidirecțională a comunicației.

S-au verificat următoarele aspecte:

– comenzile trimise din aplicație sunt recepționate corect de gateway;
– mesajele sunt propagate în rețeaua `Bluetooth Mesh` fără erori;
– răspunsurile nodurilor sunt transmise înapoi prin `MQTT`;
– interfața grafică se actualizează corect după fiecare comandă;
– nu au fost observate pierderi de mesaje în timpul testelor.

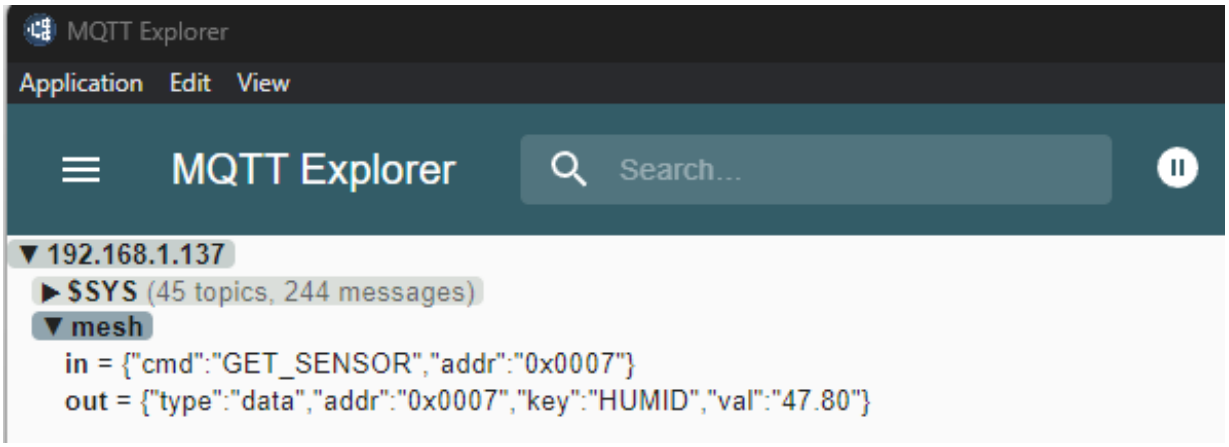


Figura 4.5. Explorer Broker MQTT

Rezultatele testelor confirmă că utilizarea gateway-ului împreună cu brokerul `MQTT` [14, 9] permite controlul centralizat al sistemului și integrarea rețelei `Bluetooth Mesh` [3] cu aplicația software.

4.5. Testarea scenariilor de automatizare

Pentru validarea funcționalității aplicației, au fost definite și testate mai multe scenarii de automatizare, cu scopul de a verifica controlul simultan și coordonat al mai multor noduri din rețea [3].

Scenariile sunt salvate local într-un fișier de configurare (`automations.json`) și sunt gestionate de aplicația software. La îndeplinirea condițiilor de declanșare, aplicația generează și transmite comenzile corespunzătoare către gateway sub formă de mesaje `MQTT` [9].

Exemple de scenarii testate:

– pornirea și oprirea aerului condiționat în funcție de temperatura raportată de senzorul din cameră [15];
– pornirea televizorului la stingerea luminii;
– scenarii bazate pe timp (*timer*), executate automat la intervale prestabilite.

În cadrul testelor s-a verificat:

– generarea corectă a comenzilor de către aplicație;
– transmiterea acestora către gateway prin `MQTT`;
– execuția corectă a comenzilor de către noduri;
– sincronizarea între starea nodurilor și interfața grafică;
– funcționarea scenariilor automate fără intervenția utilizatorului.

În toate cazurile, mesajele au fost transmise corect [9], iar nodurile au executat comenzile în ordine.

4.6. Încărcarea procesorului și utilizarea memoriei

Pentru evaluarea utilizării resurselor, s-a analizat dimensiunea firmware-ului și utilizarea memoriei pe platforma `ESP32-C3` [2], folosind informațiile generate la compilare în mediul `ESP-IDF` [5].

Pentru nodul gateway, rezultatele obținute sunt prezentate în figura de mai jos:

Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	1334916			
.text	1334916			
Flash Data	256588			
.rodata	256332			
.appdesc	256			
DRAM	162522	50.58	158774	321296
.text	99998	31.12		
.bss	47520	14.79		
.data	15004	4.67		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_reserved	24	0.29		

Total image size: 1706538 bytes (.bin may be padded larger)

Figura 4.6. Utilizarea memoriei Nod Gateway

Dimensiunea totală a imaginii firmware este de aproximativ 1.7 MB, încadrându-se în limitele memoriei flash disponibile [2].

Memoria RAM (DRAM) este utilizată în proporție de aproximativ 50%, ceea ce indică faptul că există suficient spațiu disponibil pentru funcționarea stabilă a aplicației și pentru eventuale extinderi.

În timpul funcționării, nodurile ESP32-C3 nu execută calcule intensive, majoritatea timpului fiind petrecut în așteptarea evenimentelor (mesaje Mesh, MQTT sau întreruperi hardware). Din acest motiv, încărcarea procesorului este redusă în condiții normale de utilizare.

Bluetooth Mesh utilizează memorie suplimentară pentru gestionarea mesajelor și a cheilor de securitate [3], însă în testele realizate nu au fost observate probleme legate de lipsa resurselor.

Nu au fost înregistrate blocări sau resetări ale sistemului în timpul funcționării.

Pentru partea de server, pe placa Raspberry Pi Zero 2 W [16], utilizarea resurselor a fost evaluată prin monitorizare în timpul rulării aplicației. S-a observat că utilizarea procesorului rămâne redusă (sub 10%), iar consumul de memorie pentru brokerul MQTT [14] și aplicația Flask [13] este scăzut.

Aceste rezultate confirmă faptul că sistemul poate rula stabil pe platforme *embedded* cu resurse limitate [2, 16].

4.7. Limitări ale transmisiei și latență

Pentru evaluarea performanței sistemului, s-a analizat timpul de răspuns dintre trimiterea unei comenzi și execuția acesteia la nivelul nodurilor.

Măsurarea exactă a latenței nu a fost realizată cu instrumente dedicate, însă pe baza observațiilor experimentale s-a constatat că răspunsul sistemului este rapid și adecvat pentru aplicații domotice.

În cazul nodurilor aflate în aceeași zonă, răspunsul este practic instantaneu din perspectiva utilizatorului. În situațiile în care mesajele sunt retransmise prin noduri intermediare (*relay*) sau prin gateway, poate apărea o ușoară întârziere, însă aceasta nu afectează funcționarea sistemului.

Latența este influențată de:

– numărul de noduri implicate în retransmitere [10];
– parametrii Bluetooth Mesh (*TTL*, retransmisii etc.) [3];

– timpul de procesare în gateway și aplicație [13];
– rețeaua Wi-Fi utilizată pentru MQTT [9].

În timpul testelor, nu au fost observate întârzieri semnificative sau pierderi de mesaje, iar comenzile au fost executate într-un timp suficient de mic pentru a asigura o experiență fluidă de utilizare.

Aceste rezultate indică faptul că sistemul este potrivit pentru aplicații de automatizare a locuinței, unde cerințele de latență nu sunt critice.

4.8. Fiabilitate, securitate și scalabilitate

Rețeaua Bluetooth Mesh utilizată în cadrul proiectului oferă mecanisme integrate de securitate, fiabilitate și extindere, fiind potrivită pentru aplicații domotice în care este necesară comunicarea între mai multe dispozitive distribuite [3].

Toate mesajele transmise în rețeaua *Mesh* sunt criptate folosind algoritmul AES-CCM, cu chei de rețea (NetKey) și chei de aplicație (AppKey), definite în timpul procesului de *provizionare*. Acest mecanism asigură că doar nodurile autorizate pot participa la comunicație. În timpul testelor nu au fost observate mesaje neautorizate sau comportamente anormale, confirmând funcționarea corectă a mecanismelor de securitate [3].

Fiabilitatea sistemului este asigurată prin mecanismul de retransmitere specific Bluetooth Mesh. Nodurile configurate cu rolul Relay pot retransmite mesajele primite, permițând comunicația chiar și atunci când două noduri nu se află în raza directă de acțiune. În timpul testelor s-a observat că, în cazul în care un nod intermediar este oprit, anumite mesaje nu mai ajung la destinație, ceea ce confirmă rolul important al nodurilor *relay* în asigurarea conectivității [3].

Scalabilitatea sistemului a fost verificată prin adăugarea de noi noduri în rețea folosind procesul de *provizionare* [3]. Nodurile au fost recunoscute de gateway și configurate fără a fi necesară modificarea firmware-ului existent [5]. După configurare, noile noduri au putut comunica cu restul rețelei și au putut fi controlate din aplicația software.

De asemenea, integrarea cu gateway-ul și brokerul MQTT [14, 9] permite extinderea sistemului fără modificări majore, fiind posibilă adăugarea de noi dispozitive și funcționalități prin configurare software.

4.9. Concluzii privind testarea

Testele realizate au demonstrat că sistemul dezvoltat funcționează stabil și îndeplinește cerințele definite în etapa de proiectare. Toate nodurile implementate au putut comunica în rețeaua Bluetooth Mesh [3], iar comenzile au fost transmise corect atât între noduri, cât și prin gateway.

Rețeaua Bluetooth Mesh permite controlul simultan al mai multor dispozitive și asigură transmiterea mesajelor chiar și în situațiile în care nu există legătură directă între noduri, datorită mecanismului de retransmitere [4, 10]. Gateway-ul realizat cu ESP32-C3 [2] oferă posibilitatea controlului centralizat și permite integrarea sistemului cu aplicația software prin protocolul MQTT [9].

Performanțele obținute sunt suficiente pentru aplicații domotice, timpul de răspuns fiind redus, iar funcționarea stabilă pe durata testelor. Sistemul poate fi extins prin adăugarea de noi noduri fără modificări majore ale firmware-ului, folosind procesul standard de *provizionare* și configurare.

Utilizarea unei plăci Raspberry Pi Zero 2 W [16] pentru rularea aplicației Flask [13] și a brokerului MQTT Mosquitto [14] permite funcționarea autonomă a sistemului, fără necesitatea unui calculator permanent conectat. Această abordare face soluția potrivită pentru utilizare reală în automatizarea locuințelor [1], oferind control local, fără dependență de servicii *cloud*.

În concluzie, soluția realizată confirmă că utilizarea tehnologiei Bluetooth Mesh [3] împreună cu un gateway MQTT [9] și o aplicație web locală reprezintă o variantă eficientă și flexibilă pentru implementarea unui sistem *smart home* extensibil.

Capitolul 5. Concluzii

Lucrarea a avut ca obiectiv proiectarea și implementarea unui sistem domotic bazat pe `Bluetooth Mesh` [3], utilizând microcontrolere `ESP32-C3` [2], capabil să controleze mai multe tipuri de dispozitive și să permită definirea unor scenarii de automatizare prin intermediul unei aplicații software [13].

Sistemul realizat integrează noduri cu funcționalități diferite într-o rețea de tip *mesh* [3] și oferă atât control local, cât și control centralizat prin intermediul unui gateway și al unei aplicații dezvoltate în `Python` [13].

O contribuție importantă a lucrării este realizarea unei arhitecturi complete, care funcționează integral într-o rețea locală, fără dependență de servicii *cloud* [1]. Aceasta oferă flexibilitate ridicată și permite integrarea unor dispozitive personalizate, spre deosebire de soluțiile comerciale existente.

Au fost dezvoltate mai multe tipuri de noduri hardware, fiecare având un rol specific în rețea: noduri întrerupător, noduri pentru controlul iluminatului, noduri `IR` [7], noduri senzor [15] și un nod gateway. Pentru fiecare dintre acestea au fost realizate atât partea hardware, cât și firmware-ul necesar comunicării prin `Bluetooth Mesh` [5].

Gateway-ul implementat asigură interfața între rețeaua *Mesh* și aplicația software, utilizând protocolul `MQTT` [9] pentru schimbul de date. Mesajele sunt structurate în format `JSON` [17], ceea ce permite o comunicare clară și ușor de extins între componentele sistemului.

Aplicația dezvoltată în `Flask` [13] permite controlul dispozitivelor, monitorizarea stării sistemului și definirea unor scenarii de automatizare. Acestea sunt gestionate local, fiind salvate într-un fișier de configurare și executate de aplicație, fără a necesita modificarea firmware-ului nodurilor [5].

Testele efectuate au demonstrat funcționarea stabilă a sistemului și transmiterea corectă a mesajelor în rețeaua `Bluetooth Mesh` [3], inclusiv în situații în care nodurile nu se află în raza directă de comunicație, datorită mecanismului de retransmitere [4, 10]. Timpul de răspuns obținut este adecvat pentru aplicații domotice, iar utilizarea resurselor se încadrează în limitele platformelor utilizate.

În timpul dezvoltării au fost întâmpinate dificultăți legate de configurarea rețelei `Bluetooth Mesh` și de stabilitatea anumitor plăci `ESP32-C3`. Aceste probleme au fost rezolvate prin ajustări software și alegerea unor componente hardware mai potrivite.

Pe baza rezultatelor obținute, se poate concluziona că sistemul dezvoltat îndeplinește cerințele stabilite și oferă o soluție flexibilă, scalabilă și fiabilă pentru automatizarea locuinței.

În ceea ce privește dezvoltările viitoare, sistemul poate fi extins prin adăugarea de noi tipuri de noduri [3], implementarea actualizărilor firmware *over-the-air* (`OTA`) și dezvoltarea unei aplicații mobile dedicate.

De asemenea, integrarea cu servicii externe ar putea permite controlul de la distanță, păstrând în același timp funcționarea locală.

Nu în ultimul rând, înlocuirea LED-ului IR actual cu un LED de putere de 3W, comandat printr-un tranzistor cu curent mai mare ar permite o rază de emisie semnificativ mai mare.

De asemenea, implementarea unui sistem de autentificare cu roluri diferențiate (administrator, utilizator etc.) ar permite controlul accesului la interfața web și ar crește securitatea sistemului în scenarii de utilizare reale.

În concluzie, lucrarea demonstrează că utilizarea protocolului `Bluetooth Mesh` [3] împreună cu microcontrolerele `ESP32-C3` [2] permite realizarea unui sistem domotic modern, adaptabil și potrivit pentru aplicații reale.

Bibliografie

[1] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, "A Survey on Internet of Things: Architecture, Enabling Technologies, Security and Privacy, and Applications," *IEEE Internet of Things Journal*, vol. 4, no. 5, pp. 1125–1142, Oct. 2017.

[2] Espressif Systems, "ESP32-C3 Technical Reference Manual, v1.5," Espressif Systems, Tech. Rep., 2024, Ultima accesare: Jun. 2026. https://www.espressif.com/sites/default/files/documentation/esp32-c3_technical_reference_manual_en.pdf.

[3] Bluetooth SIG, "Mesh Protocol Specification v1.1.1," Bluetooth Special Interest Group, 2023, Ultima accesare: Jun. 2026. https://www.bluetooth.com/specifications/specs/mesh-protocol-1-1-1/.

[4] ——, "Bluetooth Mesh Networking: An Introduction for Developers," Bluetooth SIG, Inc., 2017, Ultima accesare: Jun. 2026. https://www.bluetooth.com/wp-content/uploads/2019/03/Mesh-Technology-Overview.pdf.

[5] Espressif Systems, "ESP-IDF Programming Guide - Bluetooth Mesh Mode," https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-guides/esp-ble-mesh/ble-mesh-index.html, Ultima accesare: Jun. 2026.

[6] Bluetooth SIG, "Mesh Model Specification v1.1," Bluetooth Special Interest Group, 2023, Ultima accesare: Jun. 2026. https://www.bluetooth.com/specifications/specs/mesh-model-1-1/.

[7] Espressif Systems, "ESP-IDF Remote Control (RMT) API Reference," https://docs.espressif.com/projects/esp-idf/en/v5.1/esp32c3/api-reference/peripherals/rmt.html, Ultima accesare: Jun. 2026.

[8] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston, MA: Pearson Education, 2011.

[9] OASIS Standard, "MQTT Version 5.0," OASIS, Mar. 2019, Ultima accesare: Jun. 2026. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[10] K. Huang, "Managed Flooding in Wireless Mesh Networks: A Comparative Study," *IEEE Access*, vol. 11, pp. 12 450–12 462, 2023.

[11] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V Foundation, Dec. 2019.

[12] Real Time Engineers Ltd., "FreeRTOS Kernel Developer Docs," https://www.freertos.org/Documentation/RTOS_book.html, Ultima accesare: Jun. 2026.

[13] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.

[14] Eclipse Foundation, "Eclipse Mosquitto: An Open Source MQTT Broker," https://mosquitto.org/, Ultima accesare: Jun. 2026.

[15] Aosong Electronics, "AM2302 / DHT22 Product Manual," technical documentation, Tech. Rep., 2026, Ultima accesare: Jun. 2026. https://files.seeedstudio.com/wiki/Grove-Temperature_and_Humidity_Sensor_Pro/res/AM2302-EN.pdf.

[16] Raspberry Pi Ltd., “Raspberry Pi Zero 2 W Product Brief,” https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/, 2021, Ultima accesare: Jun. 2026.

[17] D. Gamble, “cJSON – Ultralightweight JSON parser in ANSI C,” GitHub repository, https://github.com/DaveGamble/cJSON, Ultima accesare: Jun. 2026.

[18] Alpha & Omega Semiconductor, “AOD4184A N-Channel Enhancement Mode Field Effect Transistor Datasheet,” Alpha & Omega Semiconductor, Tech. Rep., 2020, Ultima accesare: Jun. 2026. https://www.aosmd.com/res/datasheets/AOD4184A.pdf.

[19] Sivago Semiconductor Co., Ltd., “VS1838B Infrared Receiver Module Datasheet,” Sivago Semiconductor Co., Ltd., Tech. Rep., 2019, Ultima accesare: Jun. 2026. https://www.sivago.com.cn/upload/pdf/2022/VS1838B.pdf.

[20] J. Ganssle, “A Guide to Debouncing,” https://www.ganssle.com/debouncing.htm, Ultima accesare: Jun. 2026.